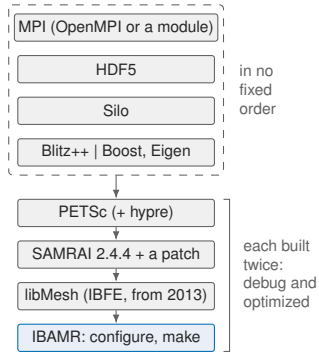
X2. HISTORICAL INSTALL RECIPES, 2011–2019

Every hand-built IBAMR install of the course and the lab, from the lab servers of 2012 to Dogwood in 2019: the commands as they were written, their versions, their slips, and what replaced them; labelled legacy

Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part X, archive • Chapter X2



Claude (Anthropic) was used to organize, combine, and format more than fourteen years (2012–2026) of the instructor's tutorial notes from courses and workshops at several universities.

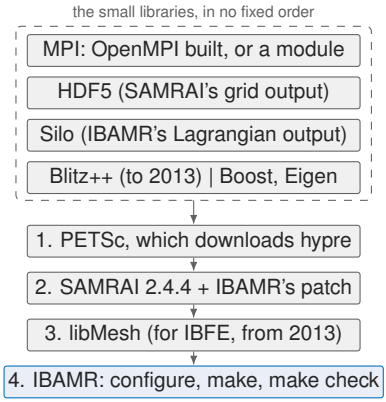
WHY KEEP THESE RECIPES, AND WHAT TO USE INSTEAD

- What this is. From 2011 to 2019 every IBAMR install was built by hand: eight to ten libraries, each configured, compiled and tested, PETSc, SAMRAI, libMesh and IBAMR usually twice. The course and the lab kept a recipe for each machine and year. This chapter keeps all of them, with the commands as written.
- Why keep them. Old papers, theses and course folders name these versions; the recipes say what a result was computed with, and why old code looks the way it does (chapter X3).
- Legacy. Every listing here is marked “historical, do not run”: many download addresses are dead, the versions (even those still on GitHub) are a decade old, and several commands have slips that this chapter points out.

> To install IBAMR today, use autoibamr (chapter 2): one script builds the whole stack, IBAMR 0.19.0 by default. To build a program, use CMake (chapter 3).

Who wrote them. The instructor’s course and lab notes (2011–2020); Boyce Griffith’s notes and e-mailed advice, summarized here (2013, 2016); Nick Battista’s KillDevil notes (2016); Alex Hoover’s module-based build (2015); notes whose author is not recorded or not verified (the Miller lab’s 2012 ReadMe and its 2012–13 forces notes and deck, the 2014 undergraduate guide, the 2019 Dogwood PDF); and two sets of Mac notes, kept unattributed.

THE STACK, AND WHAT DEPENDS ON WHAT



- The small libraries were built in no fixed order: the 2012 ReadMe built Blitz++, HDF5 and Silo before OpenMPI, the 2016–19 recipes start with Boost, and the Mavericks notes build Boost and Eigen after SAMRAI.
- PETSc, SAMRAI, libMesh (from 2013) and IBAMR always come in that order, each pointed by a path in its `configure` line at what it uses: PETSc at MPI; SAMRAI at MPI, HDF5 and Silo; libMesh at PETSc; IBAMR at all of them.
- These four are built twice, a debug build and an optimized build, and the four debug (or optimized) builds must be matched: same paths, same `PETSC_ARCH`.
- IBAMR was built with autotools: `configure`, then `make` (or `make lib`) and `make check`. A program was compiled with a Makefile that included IBAMR's `make.inc` (chapter X3).
- autoibamr (chapter 2) builds the same stack today in one command, and IBAMR itself with CMake.

All recipes of this chapter; chapter 2.

THE FOLDER LAYOUT EVERY RECIPE USED

folder	what goes there
`$HOME/sfw/linux/<pkg>/<version>`	the small libraries: OpenMPI, HDF5, Silo, Blitz++, later Boost and Eigen
`$HOME/sfw/petsc/<version>`	PETSc's source; each `PETSC_ARCH` builds in its own subfolder
`$HOME/sfw/samrai/2.4.4/`	`SAMRAI` (source), `objs-debug` (from 2016 also `objs-opt`) (builds), `linux-g++-debug`, `linux-g++-opt` (installs)
`$HOME/sfw/ibamr/`	`IBAMR` or `IBAMR2` (source), `ibamr-objs-debug` or `-dbg`, `ibamr-objs-opt` (builds)

- Variants: `/usr/bin/sfw` (root installs on THLinux, 2013), `$HOME/sfw2` (KillDevil, 2012–13), `$HOME/sfw/killdevil` (2016), `$HOME/Softwares` (the Mavericks notes, about 2015).
- A program's Makefile then pointed at the IBAMR source and build folders: `IBAMR_SRC_DIR` and `IBAMR_BUILD_DIR` (chapter X3 dates old code by these paths).
- The debug build is for finding errors: assertions on, slow. The optimized build is for runs.

The recipes of 2012–2019; the 2014–2018 course Makefiles.

EIGHT YEARS ON ONE TABLE (1/2): 2012 TO 2014

when	machine	MPI	Blitz / Boost / Eigen	HDF5	Silo	PETSc	SAMRAI patch	libMesh	IBAMR
2012	lab servers; KillDevil	OpenMPI 1.4.3 (built)	Blitz 0.9	1.8.7	4.8	3.1-p7	100922	–	SVN `trunk`
Dec 2012 – Jan 2013	lab; KillDevil	as 2012	Blitz 0.9	1.8.7	4.8	3.2-p6	111217	–	`boyceg` r3262 (forces)
April 2013	lab servers; THLinux	1.4.3, or PETSc's download	Blitz 0.9	1.8.7	4.8	3.3-p5	121212	0.8.0	`boyceg`
2013	THLinux (root); KillDevil	1.4.3 (built)	Blitz 0.9	1.8.7	4.8	3.3-p5	121212	0.8.0	`boyceg`; `ibamr-dev` r3606
Dec 2013	THLinux; a developer's Mac	1.4.3; PETSc's	Boost, Eigen (Blitz removed)	1.8.7	4.8 (4.9.1 advised)	3.3-p5	121212	0.9.1, 0.9.2	`ibamr-dev`
Feb–Mar 2014	module `ibamr/3803`	`openmpi_intel/11.1`	Boost 1.53.0	1.8.5	4.9.1	3.4	(module)	0.9.3	r3803

The SAMRAI version never changed: 2.4.4 with a patch from IBAMR's developers, whose date is its name (100922, 111217, 121212). "–": no libMesh (IB only). `boyceg` and `ibamr-dev` are branches of IBAMR's old SVN repository on Google Code.

The recipes on the next slides, compared in October 2026.

EIGHT YEARS ON ONE TABLE (2/2): 2015 TO TODAY

when	machine	MPI	Boost / Eigen	HDF5	Silo	PETSc	SAMRAI patch	libMesh	IBAMR
2015	KillDevil modules	module	Boost 1.53.0	1.8.5	4.9.1	3.4	(module)	0.9.3	GitHub, commit `386ab2`
about 2015	Mac OS X 10.9.5	OpenMPI 1.10.0	Boost 1.59.0; Eigen 3.2.6	1.8.15	4.10.2	3.5.4	121212	0.9.4	master (+ GSL)
Apr–Aug 2016	KillDevil	`openmpi_gcc/4.8.1`	Boost 1.57.0	1.8.16	4.10.2	master	121212	master	master
Aug 2016	Mac OS X 10.11.6	OpenMPI 1.10.2	Boost 1.61.0	1.8.17	4.10.2	3.7.2	121212	1.0.0	master
Sep 2016 – Apr 2017 (module: Nov 2016)	KillDevil; module `ibamr/2016-11`	`openmpi_gcc/4.8.1`	Boost 1.60.0; Eigen 3.2.7	1.8.16	4.10	3.7.2	121212	1.0.0	master
2018	Dogwood, module `ibamr/2018-03`	`openmpi_2.0.3/gcc_4.8.5`	(a module)						a snapshot from before 0.3.0
2019	Dogwood	`openmpi_2.0.3/gcc_4.8.5`	Boost 1.70.0; Eigen 3.3.7 (unused)	1.10.5	4.10.2	3.10.5	`ibamr-samrai-fixes`	1.4.1	v0.4.1
2020	Dogwood, module `ibamr/0.6.0`	`openmpi_2.0.3/gcc_4.8.5`	(a module)						0.6.0
2026	autoibamr (a test install in WSL)	system MPI	Boost 1.91.0; Eigen 3.4.0 (bundled)	1.12.2	4.11	3.23.3	IBSAMRAI2 2025.10.29	1.7.8	0.19.0 (Puma: 0.16.0)

From 2015 IBAMR came from GitHub (`git clone`; in 2019 a release tarball); in 2016 PETSc and libMesh sometimes came from their moving `master` branches. The last row is today's install (chapter 2), for comparison.

The recipes of this chapter, compared in October 2026; IBAMR's release tags; the packages of a test install of autoibamr in WSL (October 2026).

HOW TO READ AN OLD RECIPE (1/3): FIVE INVISIBLE COPY-AND-PASTE TRAPS

trap	where (examples)	what happens when you paste it
an em dash for `--`	`—-with-hypre=1` (the 2016 KillDevil notes and “Laura’s Install”); `./source/scripts/includes —link`	the option is not recognized
non-breaking spaces as indentation	the 2012 ReadMe (358 of them, on 220 lines), the 2013 forces deck and text notes, the 2016 KillDevil notes (86, one at the start of nearly every continued configure line)	the shell reads `\xa0./configure` as one unknown word (`command not found`), or hands `configure` a stray argument (`invalid host type`); the 2016 notes’ workaround, pasting through an e-mail, turned them into ordinary spaces

In this chapter the traps are shown as they were, the invisible ones marked: an em dash is printed as a red —, a curly quote as a red ”, and ␣ marks a space after a continuation backslash (other trailing spaces, harmless, are left out). Non-breaking spaces are shown as ordinary spaces. A modern editor shows them all; a slide or a word processor does not.

The recipes of this chapter, checked in their original files (October 2026).

HOW TO READ AN OLD RECIPE (2/3): THE TRAPS, CONTINUED

trap	where (examples)	what happens when you paste it
a space after the continuation backslash	`--with-c++-support \␣` (2012 PETSc); `CXX=g++-mp-4.9 \␣␣` (2016 El Capitan HDF5; a Silo line has three spaces)	the backslash escapes the space instead of the line end: the command stops there, with a stray argument (a lone space), and the lines after it run as separate commands
curly quotes	`CFLAGS="-g -O1 -Wall”` (2012 speaker notes); `--CXXOPTFLAGS="-O3“` (2013 forces deck)	an unterminated string: the shell waits for more input
soft line breaks	“Laura’s Install” (2016–17) keeps the lines of most command blocks as soft line breaks, one paragraph per block (163 soft breaks in 33 paragraphs)	readers that drop soft breaks run the commands together

The recipes of this chapter, checked in their original files (October 2026).

HOW TO READ AN OLD RECIPE (3/3): THE SLIPS THAT RECUR

kind	examples
a label that disagrees with the command	a header says PETSc 3.1-p7, the commands build 3.2-p6 (forces notes, 2012); the text names SAMRAI patch 111217, the command applies 121212 (THLinux, 2013)
the wrong folder	the SAMRAI patch run inside `SAMRAI` while the `.gz` sits one level up; an SVN checkout outside the folder the build expects; `mkdir` without `cd`
a name that changes halfway	`ibamr-objs-dbg` made, `ibamr-objs-debug` used; `sfw` for `sfw2`; `../IBAMR` for `IBAMR2`; Boost 1.60.0 in a folder named `1.6.0`
placeholders	`path/to/...`, `/path/to/libmesh-dbg`, `openmpi___`, `wget SAMRAI`
a Mac path on Linux	`--with-boost=/sw`: Fink's folder on the developer's Mac, copied into Linux recipes
root on a cluster	"Log in as root" in the KillDevil slides (right only on the lab's own THLinux servers)
duplicated options	`--download-f-blas-lapack=yes` together with `=1` or `=ifneeded`; `--without-petsc` twice
an option renamed	`--with-libmesh-compile-mode` kept after December 2013, when it became `--with-libmesh-method`
dead addresses	Google Code SVN, SourceForge SVN, Bitbucket (PETSc, Eigen), the old LLNL pages for SAMRAI and Silo, sharp-n.org
an old example path	`examples/IB/staggered_grid/explicit/ex0` (in the recipes to 2017; IBAMR itself had moved it to `examples/IB/explicit/ex0` by v0.1-rc1, May 2014)

The recipes of this chapter, compared in October 2026.

2011–13: THE MACHINES

- The lab's Ubuntu servers (two, `<host>1` and `<host>2`; 2012): packages with `sudo apt-get install`, the stack in the home folder.
- THLinux, UNC's "Tar Heel Linux" (it used `yum`, like Red Hat and Fedora): by September 2013 the same two servers ran it, with one stack built as root in `/usr/bin/sfw` for everyone (chapter X1); packages with `yum`; it also ran inside VirtualBox.
- KillDevil, UNC's cluster: the stack in the home folder (`$HOME/sfw2`), with the cluster's `openmpi_gcc` module; from 2014, IBAMR modules (slide 51).

2012 ReadMe, slide 15: "Other things to install on a new system" (one item per line on the slide) • historical, do not run

```
# "Use the command sudo apt-get install [package]"
texinfo  blas  tex  texi2html  libwxbase  m4  openmpi-dev
      ↪ mpiexec or mpirun
```

THLinux slides, slide 1: "Things to add to new THLinux" (one item per line on the slide) • historical, do not run

```
scipy ipython python-matplotlib filezilla texi2html graphviz
      ↪ screen fftw-devel
```

2012 ReadMe, slide 16: KillDevil, "Before you begin" • historical, do not run

```
export FC=gfortran CC=gcc CXX=g++ F77=gfortran
module add openmpi_gcc; module initadd openmpi_gcc
```

The Miller lab's IBAMR ReadMe (2012); the instructor's "Install THLinux and IBAMR" slides (about 2011–13); host names replaced by `<host>`.

THLINUX HOUSEKEEPING: YUM, ROOT, DROPBOX, VIRTUALBOX

"How to install on new system (installed as root)": "Instead of 'sudo-apt install <package>' use 'yum install <package>' . . . './configure' is now 'sh ./configure' . . . Had to be logged as root to 'sh ./configure'"

Dropbox on THLinux needed a repository fix written for Fedora 15: THLinux slides, slides 3–4 • historical, do not run

```
Sudo yum install nautilus-dropbox
gedit /etc/yum.repos.d/dropbox.repo
# comment out
baseurl=http://linux.dropbox.com/fedora/$releasever/
# and add
baseurl=http://linux.dropbox.com/fedora/14/
```

THLinux in VirtualBox, to share a Windows folder (December 2013): "Notes on THLinux" • historical, do not run

```
1. Devices -> Install Guest Additions
2. sudo mount -t vboxsf MyDocuments /home/<user>/Desktop/
      ↪ MyDocuments
```

- Root installs in `/usr/bin/sfw` put a software tree inside a folder meant for system programs; on a shared machine a home-folder install is safer.
- The Dropbox fix is for a release that no longer exists. Today: WSL or a Linux machine with `apt` (chapter 2).

The instructor's "Install THLinux and IBAMR" slides 2–4; "Notes on THLinux" (December 2013; user name replaced).

NEW ACCOUNTS AND DISK QUOTAS ON THE LAB'S THLINUX SERVERS

THLinux slides, slides 23–25: "Set up stuff for new accounts" • historical, do not run

```
Add users: useradd onyen
# the /home line of /etc/fstab, with usrquota,grpquota added (as root):
/dev/VolGroup00/LogVol02 /home ext3 defaults,usrquota,grpquota 1 2
mount -o remount /home
quotacheck -cugm /home
edquota username
# quota -u <user> # Check quota for a user
# repquota /home # Full report for the partition for all users
Default amounts:
50000000 for grad student
25000000 for ugrad
```

- An account per student (named by the student's UNC "onyen"), and a disk quota on `/home` so that one IBAMR install could not fill the disk.
- `edquota` opens a user's limits in an editor; the slides' defaults were 50000000 for a graduate student and 25000000 for an undergraduate, in `edquota`'s 1 KB blocks (about 50 GB and 25 GB).
- In `# quota -u <user>` and `# repquota /home` the first `#` is the slide's root prompt, not a comment.
- The same slides add two pages of `vi` commands (`:q`, `:x`, `i I a A o O`).
- Today a cluster account comes from the university (chapter 1); quotas are the cluster's.

The instructor's "Install THLinux and IBAMR" slides 23–25 (a user name in the `quota` example replaced by `<user>`).

BLITZ++ 0.9 AND ITS TWO PATCHES (2012–13)

2012 ReadMe, slide 17 • historical, do not run

```
cd $HOME/sfw/linux
tar xvfz blitz-0.9.tar.gz
 cd blitz-0.9
 ./configure \
   CC=gcc \
   CXX=g++ \
   F77=gfortran \
   FC=gfortran \
   --prefix=$HOME/sfw/linux/blitz/0.9 \
   --enable-optimize \
   --disable-debug \
   --enable-shared
cd blitz
wget http://www.sharp-n.org/patches/mathfunc.h.patch
wget http://www.sharp-n.org/patches/funcs.h.patch
patch funcs.h funcs.h.patch
patch mathfunc.h mathfunc.h.patch
cd ..
 make
 make check-testsuite
 make install
```

- Blitz++ gave IBAMR its array classes (`TinyVector` for positions) until December 2013, when IBAMR replaced it with Boost and Eigen (slide 45).
- The two patches came from a third-party web site that is gone.
- A slip in the KillDevil and THLinux slides: `--prefix=$HOME/sfw2/linux/blitz-0.9\` has no space before the backslash, so the next option is glued onto the prefix, and the folder differs from the one IBAMR is later told to use, `--with-blitz=$HOME/sfw2/linux/blitz/0.9`.
- The indentation of the 2012 lines is made of non-breaking spaces, mixed with ordinary ones (slide 7).

The Miller lab's IBAMR ReadMe (2012), slide 17; the instructor's KillDevil slide 4 and THLinux slide 6.

HDF5 1.8.7, SILO 4.8 AND OPENMPI 1.4.3 (2012–13)

2012 ReadMe, slide 18 • historical, do not run

```
cd $HOME/sfw/linux
tar xvfz hdf5-1.8.7.tar.gz
 cd hdf5-1.8.7
 ./configure \
   CC=gcc \
   CXX=g++ \
   FC=gfortran \
   F77=gfortran \
   --enable-production \
   --disable-debug \
   --prefix=$HOME/sfw/linux/hdf5/1.8.7
 make
 make check
 make install
```

slide 19 • historical, do not run

```
cd $HOME/sfw/linux
tar xvfz silo-4.8.tar.gz
 cd silo-4.8
 ./configure \
   CC=gcc \
   CXX=g++ \
   FC=gfortran \
   F77=gfortran \
   --prefix=$HOME/sfw/linux/silo/4.8 \
   --disable-silex
 make
 make install
```

slide 20 • historical, do not run

```
cd $HOME/sfw/linux
tar xvfj openmpi-1.4.3.tar.bz2
 cd openmpi-1.4.3
 ./configure \
   CC=gcc \
   CXX=g++ \
   FC=gfortran \
   F77=gfortran \
   --prefix=$HOME/sfw/linux/openmpi/1.4.3 \
   --disable-mpi-cxx-seek \
   --disable-heterogeneous \
   --enable-orterun-prefix-by-default
 make
 make check
 make install
```

The three quiet builds: HDF5 is the format of SAMRAI's VisIt files (the grid data); Silo is the library IBAMR writes its Lagrangian data with, in Silo's own PDB format, and no recipe gives Silo `--with-hdf5` (`--disable-silex` skips Silo's file browser); OpenMPI gives `mpicc`, `mpicxx`, `mpif90` and `mpirun`. The KillDevil and THLinux slides are the same with `$HOME/sfw2` or `/usr/bin/sfw`, and `sh ./configure` for Silo (both) and OpenMPI (THLinux); their OpenMPI slide is labelled "opempi-1.4.3".

The Miller lab's IBAMR ReadMe (2012), slides 18–20; the instructor's KillDevil slides 5–7.

PETSC 3.1-P7 (2012) (1/2): UNPACK, AND THE DEBUG BUILD

2012 ReadMe, slides 21–22 (␣: a space after the backslash) • historical, do not run

```
 cd $HOME/sfw
 mkdir petsc
 cd petsc
 tar xvfz petsc-3.1-p7.tar.gz
 mv petsc-3.1-p7 3.1-p7
 cd 3.1-p7
# Configure debugging petsc
export PETSC_DIR=$PWD
 export PETSC_ARCH=linux
 ./config/configure.py \
   --CC=$HOME/sfw/linux/openmpi/1.4.3/bin/mpicc \
   --CXX=$HOME/sfw/linux/openmpi/1.4.3/bin/mpicxx \
   --FC=$HOME/sfw/linux/openmpi/1.4.3/bin/mpif90 \
   --LDFLAGS="-L$HOME/sfw/linux/openmpi/1.4.3/lib -Wl,-rpath,$HOME/sfw/
      ↪ linux/openmpi/1.4.3/lib" \
   --with-default-arch=0 \
   --PETSC_ARCH=$PETSC_ARCH \
   --with-debugging=1 \
   --with-c++-support \␣
  --download-f-blas-lapack=yes \
  --download-f-blas-lapack=ifneeded \
   --with-hypre=1 \
   --download-hypre=1
 make
 make test
```

- `PETSC_DIR` is the source folder, `PETSC_ARCH` names one build of it: `linux` (debug) and `linux-opt` (optimized) live side by side in the same source tree.
- PETSc is compiled with the OpenMPI just built (`--CC`, `--CXX`, `--FC`), and the `-rpath` makes the libraries find it at run time.
- `--download-hypre=1`: PETSc fetches and builds hypre, the multigrid library IBAMR's solvers use; later recipes keep this.
- `--download-f-blas-lapack` is given twice, with `yes` and with `ifneeded`.
- `make test` runs PETSc's own small examples.

PETSC 3.1-P7 (2012) (2/2): THE OPTIMIZED BUILD, AND THE TRAP IN BOTH

The optimized build (slide 23) is the debug configure with four changes: 2012 ReadMe, slide 23, the lines that differ • historical, do not run

```
export PETSC_ARCH=linux-opt
  --COPTFLAGS="-O3" \
  --CXXOPTFLAGS="-O3" \
  --FOPTFLAGS="-O3" \
  --with-debugging=0 \
  --with-x=0
```

The `-O3` lines follow `--FC`; `--with-x=0` is the last line; the `--with-c++-support \␣` line is the same as in the debug build.

The trap. In both slides the line `--with-c++-support \` ends with a space after the backslash. The backslash then escapes the space (configure gets one extra argument, a lone space) instead of the line end, so the configure command ends there, before the BLAS/LAPACK and hypre options. The lines after it (four in the debug build, five in the optimized), still joined by their own backslashes, run as one separate command and fail: `--download-f-blas-lapack=yes: command not found`. Even if PETSc's configure then succeeds, it has no hypre, and IBAMR's `--with-hypre` later finds nothing.

The cure, then and now: no character after a continuation backslash. A modern editor shows trailing spaces; today autoibamr writes these lines itself.

The Miller lab's IBAMR ReadMe (2012), slides 22–23.

SAMRAI 2.4.4 (2012): UNPACK AND PATCH

2012 ReadMe, slide 24 • historical, do not run

```
cd $HOME/sfw
mkdir samrai
cd samrai
mkdir 2.4.4
cd 2.4.4
tar xvfz SAMRAI-v2.4.4.tar.gz
# "Next, patch SAMRAI using the patch file
#  SAMRAI-v2.4.4-patch-100922.gz:"
cd SAMRAI
./source/scripts/includes --link
gunzip -c ../SAMRAI-v2.4.4-patch-100922.gz | patch -p2
```

slide 25, the build folder • historical, do not run

```
cd $HOME/sfw/samrai/2.4.4
mkdir objs-debug
cd objs-debug
# slide 26 (optimized) has no mkdir or cd:
#   it is configured in objs-debug too
```

- SAMRAI is LLNL's adaptive mesh library; IBAMR used version 2.4.4, with patches from IBAMR's developers, throughout these recipes, and still does as their fork IBSAMRAI2, which autoibamr builds.
- `includes --link` makes SAMRAI's header links; the patch is piped through `gunzip` into `patch -p2` from inside `SAMRAI`, so the `.gz` must sit one folder up: `../`.
- The patch changed with the date: 100922 (2012), 111217 (the forces branch), 121212 (2013–2017, later attached to IBAMR's v0.1-rc1 release on GitHub), `ibamr-samrai-fixes.patch` (2019).
- SAMRAI is configured in a build folder beside its source, `objs-debug` (the optimized build reused it until 2016; from August 2016 it has its own `objs-opt`), and installed in another (`linux-g++-debug`, `linux-g++-opt`): next slides.

The Miller lab's IBAMR ReadMe (2012), slides 24–26; the later recipes of this chapter.

SAMRAI'S CONFIGURE, FLAG BY FLAG (1/5): THE LIST USED 2012–2019

"Laura's Install" (2016), the debug build • historical, do not run

```
../SAMRAI-2.4.4/configure \
  --prefix=$HOME/sfw/samrai/2.4.4/linux-g++-debug \
  --with-CC=gcc \
  --with-CXX=g++ \
  --with-F77=gfortran \
  --with-MPICC=mpicc \
  --with-hdf5=$HOME/sfw/linux/hdf5/1.8.16 \
  --without-hypre \
  --with-silo=$HOME/sfw/linux/silo/4.10 \
  --without-blaslapack \
  --without-cubes \
  --without-eleven \
  --without-kinsol \
  --without-petsc \
  --without-sundials \
  --without-x \
  --with-doxygen \
  --with-dot \
  --enable-debug \
  --disable-opt \
  --enable-implicit-template-instantiation \
  --disable-deprecated
make
make install
```

- Nearly the same option list appears in every recipe of this chapter, from the 2012 ReadMe to the 2019 Dogwood PDF: the paths, the compilers and the versions change (slides 20–22), and two recipes change the list itself (the 2016 KillDevil notes leave out `--with-silo` and add `-g -O0` flags to the debug build; the 2019 PDF lists `--without-petsc` twice).
- `--prefix`: where `make install` puts SAMRAI; IBAMR's `--with-samrai` points here.
- `--with-CC`, `--with-CXX`, `--with-F77`, `--with-MPICC`: the compilers. In 2012–13 `--with-MPICC` was the home-built OpenMPI's `mpicc`; in 2019 the compilers themselves were `mpicc`, `mpicxx`, `mpifort`.
- `--with-hdf5`, `--with-silo`: the two output libraries built before.

"Laura's Install" (2016–17); the other recipes of 2012–2019, compared in October 2026.

SAMRAI'S CONFIGURE, FLAG BY FLAG (2/5): WHAT IS SWITCHED OFF, AND THE OPTIMIZED BUILD

- The `--without-...` options switch off SAMRAI's own links to solvers (hypre, PETSc, SUNDIALS, KINSOL, BLAS/LAPACK) and optional outside packages (the CUBES and Eleven embedded-boundary libraries, and X11): IBAMR reaches PETSc and hypre itself.
- `--with-doxygen`, `--with-dot`: SAMRAI's documentation tools.
- `--enable-implicit-template-instantiation` and `--disable-deprecated`: as IBAMR's own build instructions asked.
- `--enable-debug --disable-opt` make the debug build.

The optimized build adds `CFLAGS="-O3" CXXFLAGS="-O3" FFLAGS="-O3"` before `--prefix`, installs into `.../linux-g++-opt`, and has `--disable-debug --enable-opt` in place of `--enable-debug --disable-opt`; the rest of the list is the same.

Then `make` and `make install`, in each build folder.

"Laura's Install" (2016–17); the other recipes of this chapter.

SAMRAI'S CONFIGURE, FLAG BY FLAG (3/5): HOW THE STEP VARIED, 2012–2013

recipe	patch, and how it was applied	installed in (debug, optimized)	MPI compiler	HDF5 / Silo
2012 ReadMe	100922: `cd SAMRAI; ./source/scripts/includes --link; gunzip -c ../SAMRAI-v2.4.4-patch-100922.gz | patch -p2`; configured from `objs-debug`, after `export PETSC_ARCH=linux` (or `linux-opt`)	`$HOME/sfw/samrai/2.4.4/linux-g++-{debug,opt}`	the home-built OpenMPI 1.4.3's `mpicc`	1.8.7 / 4.8
forces notes (2012–13)	111217: `gunzip -c ../SAMRAI-v2.4.4-patch-111217.gz | patch -p2`; `./configure` in the source folder	optimized only: `$HOME/sfw/SAMRAI_petsc3p2/linux-g++-opt`	as 2012	1.8.7 / 4.8
"InstallNotes2" (2013)	111217; `./configure` in `$HOME/sfw/samrai_petsc3p2/2.4.4/SAMRAI`	optimized only: `$HOME/sfw/samrai_petsc3p2/linux-g++-opt`	as 2012	1.8.7 / 4.8
KillDevil (2012–13)	121212: `cd SAMRAI; ./source/scripts/includes --link; gunzip -c SAMRAI-v2.4.4-patch-121212.gz | patch -p2`, from the wrong folder (the `.gz` is one level up)	`$HOME/sfw2/samrai/2.4.4/linux-g++-{debug,opt}`	`$HOME/sfw2/linux/openmpi/1.4.3/bin/mpicc`	1.8.7 / 4.8
THLinux (2013)	the text says 111217, the command applies 121212 (the same wrong-folder command)	`/usr/bin/sfw/samrai/2.4.4/linux-g++-{debug,opt}`	`/usr/bin/sfw/linux/openmpi/1.4.3/bin/mpicc`	1.8.7 / 4.8
"THLinuxInstall" (April 2013)	names `SAMRAI-v2.4.4-patch-121212.gz` only	`$HOME/sfw/samrai/2.4.4/linux-g++-{debug,opt}`	PETSc's: `$HOME/sfw/petsc/3.3-p5/linux/bin/mpicc`	1.8.7 / 4.8
December 2013 notes	121212, fixed: `gunzip -c ../SAMRAI-v2.4.4-patch-121212.gz | patch -p2`	as THLinux	as THLinux	1.8.7 / 4.8

The KillDevil, THLinux and December 2013 optimized configures write `CXXFLAGS="-O3" FFLAGS="-O3" \` on one line (harmless).

Each recipe, checked in its original (October 2026).

SAMRAI'S CONFIGURE, FLAG BY FLAG (4/5): HOW THE STEP VARIED, 2015–2016

recipe	patch, and how it was applied	installed in (debug, optimized)	compilers	HDF5 / Silo
Mavericks notes (about 2015)	121212 from IBAMR's GitHub release: `wget https://github.com/IBAMR/IBAMR/releases/download/v0.1-rc1/SAMRAI-v2.4.4-patch-121212.gz`; `gunzip -c $HOME/Softwares/SAMRAI/2.4.4/SAMRAI-v2.4.4-patch-121212.gz | patch -p2`; `../configure` from `2.4.4/linux-dbg`	debug only: `$HOME/Softwares/SAMRAI/2.4.4/linux-dbg`	`gcc-mp-4.9`, `g++-mp-4.9`, `gfortran-mp-4.9`; `$HOME/Softwares/OpenMPI/1.10.0/bin/mpicc`	1.8.15 / 4.10.2
KillDevil, 2016 (Nick Battista)	121212, with an extra `tar xvfz` of the patch (slide 64); `../SAMRAI/configure`	`$HOME/sfw/samrai/2.4.4/killdevil-gcc-{opt,dbg}`	`gcc`, `g++`, `gfortran`; `mpicc`	1.8.16 (in `$HOME/sfw/killdevil`) / none
El Capitan (August 2016)	121212 from the same GitHub release; `cd SAMRAI-2.4.4/; ./source/scripts/includes --link; gunzip -c ../SAMRAI-v2.4.4-patch-121212.gz | patch -p2`	`$HOME/sfw/samrai/2.4.4/linux-g++-{debug,opt}`	`gcc-mp-4.9` …; `$HOME/sfw/linux/openmpi/1.10.2/bin/mpicc`	1.8.17 / 4.10.2

The 2016 KillDevil notes build the debug SAMRAI with `CFLAGS="-g -O0" CXXFLAGS="-g -O0" FFLAGS="-g -O0"` and leave out `--with-silo`.

Each recipe, checked in its original (October 2026).

SAMRAI'S CONFIGURE, FLAG BY FLAG (5/5): HOW THE STEP VARIED, 2016–2019

recipe	patch, and how it was applied	installed in (debug, optimized)	compilers	HDF5 / Silo
"Laura's Install" (2016–17)	`tar xvfz SAMRAI-v2.4.4.tar.gz; mv SAMRAI SAMRAI-2.4.4; cd SAMRAI-2.4.4; ./source/scripts/includes --link;` "*Put patch in SAMRAI-2.4.4 folder*" `gunzip -c SAMRAI-v2.4.4-patch-121212.gz | patch -p2`	as quoted on slide 18	`gcc`, `g++`, `gfortran`; `mpicc`	1.8.16 / 4.10
Dogwood (2019)	a new patch: `cp $HOME/ibamr-samrai-fixes.patch .; patch -p1 < ibamr-samrai-fixes.patch`	`$HOME/sfw/samrai/2.4.4/linux-g++-{debug,opt}`	`mpicc`, `mpicxx`, `mpifort` (no `--with-MPICC`)	1.10.5 / 4.10.2

The patch changed for the last time in 2019: `patch -p1` with a plain patch file, in place of the dated `.gz` piped through `gunzip`. The 2019 debug line lists `--without-petsc` twice and has no backslashes (slide 96). Today autoibamr builds IBSAMRAI2, IBAMR's fork of SAMRAI 2.4.4 with the patches applied.

Each recipe, checked in its original (October 2026).

IBAMR FROM GOOGLE CODE (2012) (1/3): GET IT, AND THE DEBUG BUILD

2012 ReadMe, slides 27–28 • historical, do not run

```
cd $HOME/sfw
 mkdir ibamr
 cd ibamr
 svn co https://ibamr.googlecode.com/svn/trunk IBAMR
# configure and compile a debugging build of IBAMR
 cd $HOME/sfw/ibamr
 mkdir ibamr-objs-debug
 cd ibamr-objs-debug
 export PETSC_ARCH=linux
 export PETSC_DIR=$HOME/sfw/petsc/3.1-p7
 ../IBAMR/configure --config-cache \
   CFLAGS="-g -O1 -Wall" \
   CXXFLAGS="-g -O1 -Wall" \
   FFLAGS="-g -O1 -Wall" \
   FCFLAGS="-g -O1 -Wall" \
```

slide 28, continued • historical, do not run

```
   CC=$HOME/sfw/linux/openmpi/1.4.3/bin/mpicc \
   CXX=$HOME/sfw/linux/openmpi/1.4.3/bin/mpicxx \
   F77=$HOME/sfw/linux/openmpi/1.4.3/bin/mpif90 \
   FC=$HOME/sfw/linux/openmpi/1.4.3/bin/mpif90 \
   MPICC=$HOME/sfw/linux/openmpi/1.4.3/bin/mpicc \
   MPICXX=$HOME/sfw/linux/openmpi/1.4.3/bin/mpicxx \
   CPPFLAGS="-DOMPI_SKIP_MPICXX" \
   --with-hypre=$PETSC_DIR/$PETSC_ARCH \
   --with-samrai=$HOME/sfw/samrai/2.4.4/linux-g++-debug \
   --with-hdf5=$HOME/sfw/linux/hdf5/1.8.7 \
   --with-blitz=$HOME/sfw/linux/blitz/0.9 \
   --with-silo=$HOME/sfw/linux/silo/4.8
make
make check
```

The Miller lab's IBAMR ReadMe (2012), slides 27–28.

IBAMR FROM GOOGLE CODE (2012) (2/3): WHAT THE CONFIGURE LINE SAYS

- IBAMR lived in a Subversion repository on Google Code; `trunk` was its main line. Google Code closed in January 2016; IBAMR's first GitHub release, v0.1-rc1, is from May 2014.
- `--config-cache` lets a second configure reuse the first one's answers.
- `CFLAGS`, `CXXFLAGS`, `FFLAGS`, `FCFLAGS`: `-g -O1 -Wall` for the debug build (symbols, light optimization, the common warnings).
- `CC`, `CXX`, `F77`, `FC`, `MPICC`, `MPICXX`: the home-built OpenMPI's wrappers, the same ones PETSc was built with.
- `CPPFLAGS="-DOMPI_SKIP_MPICXX"` keeps OpenMPI's C++ bindings out of the build, as most later recipes do.
- `--with-hypre=$PETSC_DIR/$PETSC_ARCH`: the hypre that PETSc downloaded into its debug build.
- `--with-samrai`, `--with-hdf5`, `--with-blitz`, `--with-silo`: the install folders of slides 13–18.
- IBAMR was configured in a build folder beside the source (`ibamr-objs-debug`), not in the source itself; only the 2016 KillDevil notes configure in the source tree (slide 65).

IBAMR FROM GOOGLE CODE (2012) (3/3): THE OPTIMIZED BUILD AND THE FIRST RUN

2012 ReadMe, slide 29: what differs from the debug build • historical, do not run

```
mkdir ibamr-objs-opt
cd ibamr-objs-opt
export PETSC_ARCH=linux-opt
# CFLAGS, CXXFLAGS, FFLAGS, FCFLAGS = "-O3 -Wall"
  --with-samrai=$HOME/sfw/samrai/2.4.4/linux-g++-opt \
# then make, make check as before
```

slide 30: “You may now compile and run the IBAMR examples, e.g.,” • historical, do not run

```
cd $HOME/sfw/ibamr/ibamr-objs-debug/examples/IB/staggered_grid/
    ↪ explicit/ex0
make examples
./main2d input2d
```

speaker notes of slide 28 (curly closing quotes; shortened) • historical, do not run

```
../IBAMR/configure --config-cache CFLAGS="-g -O1 -Wall” CXXFLAGS
    ↪ ="-g -O1 -Wall” FFLAGS="-g -O1 -Wall” FCFLAGS="-g -O1 -
    ↪ Wall” ... CPPFLAGS="-DOMPI_SKIP_MPICXX”
```

- In the optimized build the four flag lines come after the `MPICXX` line; otherwise it is the debug configure.
- `make check` adds no tests in IBAMR’s autotools build: like `make lib` (`lib: all`), it builds the libraries. `make examples` in an example’s build folder compiles that example.
- `ex0` of `IB/staggered_grid/explicit` was the first example to run; today it is `examples/IB/explicit/ex0` in IBAMR’s source, and this series starts with its own examples (chapter 3).
- The speaker notes held a one-line version of every configure command, handy for pasting except that the closing quotes of slide 28’s note are curly (slide 7) and a non-breaking space glues its `FC=...mpif90` to `MPICC=....`
- Running on KillDevil (`bsub`, `/netscr`) is in chapter X1.

The Miller lab’s IBAMR ReadMe (2012), slides 28–31 and speaker notes.

THE “FORCES” BRANCH (DECEMBER 2012 – JANUARY 2013) (1/7): PETSC 3.2-P6

A second recipe, of December 2012 – January 2013, built IBAMR’s `boyceg` branch at revision 3262, for the lab’s work on forces, in optimized form only. It reused Blitz++, HDF5, Silo and OpenMPI from the 2012 recipe and rebuilt PETSc and SAMRAI.

“InstallationNotesForces.txt” (each long command is one line in the file) • historical, do not run

```
**PETSC 3.1-p7*******************************
cd $HOME/sfw/petsc
tar xvfz petsc-3.2-p6.tar.gz
cd petsc-3.2-p6
//CONFIGURE OPTIMIZED PETSC
export PETSC_DIR=$PWD
export PETSC_ARCH=linux-opt
./config/configure.py --CC=$HOME/sfw/linux/openmpi/1.4.3/bin/mpicc --CXX=$HOME/sfw/linux/openmpi/1.4.3/bin/mpicxx --FC=$HOME/sfw/
      ↪ linux/openmpi/1.4.3/bin/mpif90 --COPTFLAGS="-O3" --CXXOPTFLAGS="-O3" --FOPTFLAGS="-O3" --LDFLAGS="-L$HOME/sfw/linux/openmpi
      ↪ /1.4.3/lib -Wl,-rpath,$HOME/sfw/linux/openmpi/1.4.3/lib" --with-default-arch=0 --PETSC_ARCH=$PETSC_ARCH --with-debugging=0
      ↪ --with-c++-support --download-f-blas-lapack=yes --download-f-blas-lapack=ifneeded --with-hypre=1 --download-hypre=1 --with-
      ↪ x=0
make
make test
```

The header says 3.1-p7; every command builds 3.2-p6. The lines starting with `//` are notes, not shell comments: pasted into a shell, each gives an error such as `//CONFIGURE: No such file or directory`.

The Miller lab’s forces install notes (text file, December 2012 – January 2013).

THE “FORCES” BRANCH (2/7): SAMRAI WITH PATCH 111217

“InstallationNotesForces.txt”, continued • historical, do not run

```
**SAMRAI 2.4.4**********************************
 cd $HOME/sfw
 tar xvfz SAMRAI-v2.4.4.tar.gz
mv SAMRAI SAMRAI_petsc3p2
# "Next, patch SAMRAI using the patch file SAMRAI-v2.4.4-patch-111217.gz:"
cd SAMRAI_petsc3p2
./source/scripts/includes --link
gunzip -c ../SAMRAI-v2.4.4-patch-111217.gz | patch -p2
export PETSC_ARCH=linux
//OPTIMIZED SAMRAI
./configure CFLAGS="-O3" CXXFLAGS="-O3" FFLAGS="-O3" --prefix=$HOME/sfw/SAMRAI_petsc3p2/linux-g++-opt --with-CC=gcc --with-CXX=g++ --
      ↪ with-F77=gfortran --with-MPICC=$HOME/sfw/linux/openmpi/1.4.3/bin/mpicc --with-hdf5=$HOME/sfw/linux/hdf5/1.8.7 --without-
      ↪ hypre --with-silo=$HOME/sfw/linux/silo/4.8 --without-blaslapack --without-cubes --without-eleven --without-kinsol --without
      ↪ -petsc --without-sundials --without-x --with-doxygen --with-dot --disable-debug --enable-opt --enable-implicit-template-
      ↪ instantiation --disable-deprecated
make
make install
```

- A separate SAMRAI (`SAMRAI_petsc3p2`) beside the 2012 one, patched with 111217, and configured inside its own source folder.
- `PETSC_ARCH=linux` before an optimized SAMRAI: the forces deck adds “Note: PETSC_ARCH=linux should perhaps be PETSC_ARCH=linux-opt”. It does no harm here: SAMRAI is configured `--without-petsc`.
- A second copy of the file differs only by three stray non-ASCII bytes before the three patch lines.

THE “FORCES” BRANCH (3/7): IBAMR AT REVISION 3262

“InstallationNotesForces.txt”, continued • historical, do not run

```
**IBAMR FORCES**********************
cd $HOME/sfw/
mkdir ibamr_forces
svn co https://ibamr.googlecode.com/svn/branches/boyceg IBAMR
cd IBAMR
svn up -r3262
cd $HOME/sfw/ibamr_forces
mkdir ibamr-objs-opt
cd ibamr-objs-opt
export PETSC_ARCH=linux-opt
export PETSC_DIR=$HOME/sfw/petsc/petsc-3.2-p6
../IBAMR/configure --config-cache CC=$HOME/sfw/linux/openmpi/1.4.3/bin/mpicc CXX=$HOME/sfw/linux/openmpi/1.4.3/bin/mpicxx F77=$HOME/
      ↪ sfw/linux/openmpi/1.4.3/bin/mpif90 FC=$HOME/sfw/linux/openmpi/1.4.3/bin/mpif90 MPICC=$HOME/sfw/linux/openmpi/1.4.3/bin/
      ↪ mpicc MPICXX=$HOME/sfw/linux/openmpi/1.4.3/bin/mpicxx CFLAGS="-O3 -Wall" CXXFLAGS="-O3 -Wall" FFLAGS="-O3 -Wall" FCFLAGS="-
      ↪ O3 -Wall" CPPFLAGS="-DOMPI_SKIP_MPICXX" --with-hypre=$PETSC_DIR/$PETSC_ARCH --with-samrai=$HOME/sfw/SAMRAI_petsc3p2/linux-g
      ↪ ++-opt --with-hdf5=$HOME/sfw/linux/hdf5/1.8.7 --with-blitz=$HOME/sfw/linux/blitz/0.9 --with-silo=$HOME/sfw/linux/silo/4.8
      ↪ --enable-large-ghost-cell-width --enable-expensive-cf-interpolation
make
make check
```

The optimized build only, against the SAMRAI of the last slide and the 2012 Blitz++, HDF5 and Silo; then `make` and `make check`.

The Miller lab’s forces install notes (December 2012 – January 2013).

THE “FORCES” BRANCH (4/7): WHAT THE BUILD ADDS, AND A SLIP

- `svn up -r3262` pins the branch to one revision. `--enable-large-ghost-cell-width` and `--enable-expensive-cf-interpolation` are the two options this build adds (by their names: wider ghost-cell regions, and a costlier interpolation at coarse–fine boundaries).
- A slip: `mkdir ibamr_forces` but no `cd`: the checkout lands in `$HOME/sfw/IBAMR`, while the build in `$HOME/sfw/ibamr_forces/ibamr-objs-opt` looks for `../IBAMR`. The later text version adds the `cd` (slide 31).

The Miller lab’s forces install notes (December 2012 – January 2013).

THE “FORCES” BRANCH (5/7): THE SECOND TEXT VERSION, PETSC AND SAMRAI

“InstallNotes2.txt” (2013), PETSc; blank lines left out; “...”: as in the first file • historical, do not run

```
PETSC3.2-p6
cd $HOME/sfw
mkdir petsc3p2
cd petsc3p2
tar xvfz petsc-3.2-p6.tar.gz
mv petsc-3.2-p6 3.2-p6
cd 3.2-p6
export PETSC_DIR=$PWD
export PETSC_ARCH=linux-opt
./config/configure.py ... --with-c++-support --download-f-blas-
      ↪ lapack=yes --download-f-blas-lapack=1 --with-hypre=1 --
      ↪ download-hypre=1 --with-x=0
make
make test
```

SAMRAI • historical, do not run

```
//SAMRAI2.4.4
cd $HOME/sfw
mkdir samrai_petsc3p2
cd samrai_petsc3p2
 mkdir 2.4.4
 cd 2.4.4
 tar xvfz SAMRAI-v2.4.4.tar.gz
# "Next, patch SAMRAI using the patch file SAMRAI-v2.4.4-patch
      ↪ -111217.gz:"
 cd SAMRAI
 ./source/scripts/includes --link
 gunzip -c ../SAMRAI-v2.4.4-patch-111217.gz | patch -p2
cd $HOME/sfw/samrai_petsc3p2/2.4.4/SAMRAI
export PETSC_ARCH=linux
./configure ... --prefix=$HOME/sfw/samrai_petsc3p2/linux-g++-
      ↪ opt ...
make
make install
```

The Miller lab’s “InstallNotes2.txt” (2013).

THE “FORCES” BRANCH (6/7): THE SECOND TEXT VERSION, IBAMR, AND WHAT CHANGED

“InstallNotes2.txt” (2013), IBAMR • historical, do not run

```
// IBAMR FORCES
cd $HOME/sfw/
mkdir ibamr_forces
cd ibamr_forces
svn co https://ibamr.googlecode.com/svn/branches/boyceg IBAMR
cd IBAMR
svn up -r3262
cd $HOME/sfw/ibamr_forces
mkdir ibamr-objs-opt
cd ibamr-objs-opt
export PETSC_ARCH=linux-opt
export PETSC_DIR=$HOME/sfw/petsc3p2/3.2-p6
../IBAMR/configure ... --with-samrai=$HOME/sfw/samrai_petsc3p2/linux-
      ↪ g++-opt ...
make
make check
```

Against the first file

- the header names the right PETSc, 3.2-p6, unpacked into `petsc3p2/3.2-p6`;
- SAMRAI is unpacked into `samrai_petsc3p2/2.4.4` and keeps the name `SAMRAI`;
- the PETSc line has `--download-f-blas-lapack=1` for `=ifneeded`;
- `cd ibamr_forces` is there, so the checkout lands where the build looks for it (slide 29);
- the headers are `PETSC3.2-p6`, `//SAMRAI2.4.4` and `// IBAMR FORCES`, in place of three starred headers and two `//` notes; pasted, each still gives an error;
- six lines begin with a non-breaking space byte, shown here as spaces.

The Miller lab’s “InstallNotes2.txt” (2013) and “InstallationNotesForces.txt”.

THE “FORCES” BRANCH (7/7): THE SLIDE DECK OF JANUARY 2013

The forces install deck (14 slides, January 2013) shows the same steps:

- slide 2: `export FC=gfortran CC=gcc CXX=g++ F77=gfortran` and `module initadd openmpi_gcc/4.4.6` on KillDevil;
- slides 3–6: Blitz++, HDF5, Silo, OpenMPI as in 2012;
- slides 8, 10, 12: the PETSc, SAMRAI and IBAMR configures written over many lines without continuation backslashes, with `--CXXOPTFLAGS="-O3“` (a curly quote) and `--download-f-blas-lapack=yes` with `=1`; pasted line by line, every option runs as a command;
- slide 13 runs `ex0` from `ibamr-objs-debug`, which this recipe never builds.

The speaker notes hold one-line versions of the configures: those of slides 10 and 12 match the first text file, but the PETSc note (slide 8) has `--download-f-blas-lapack=1`, as in “InstallNotes2.txt”, where the first file has `=ifneeded`; slides 3–6 carry one-line notes for Blitz++, HDF5, Silo and OpenMPI, which the text file does not have.

The Miller lab’s forces install deck (January 2013; author not verified).

SPRING 2013 (1/3): PETSC 3.3-P5 AND LIBMESH 0.8.0, FOR IBFE

MATH 769 “IBFE install” slides 2–4 (April 2013) • historical, do not run

```
# "Make clean/make distclean ibamr, samrai, and petsc (or delete them).
#  Install petsc 3.3-p5. Install samrai, updating the petsc version.
#  Now install libmesh version 0.8.0."
# Install a debugging version of libmesh
svn co  https://libmesh.svn.sourceforge.net/svnroot/libmesh/tags/libmesh
      ↪ -0.8.0/libmesh$HOME/sfw//libmesh-dbg
    cd /path/to/libmesh-dbg
    ./configure \
       PETSC_DIR=$HOME/sfw/petsc/3.3-p5 \
       PETSC_ARCH=linux \
       CC=gcc \
       CXX=g++ \
       FC=gfortran \
       F77=gfortran \
       --enable-everything --disable-shared --disable-exceptions
    make METHOD=dbg
# slide 4, optimized: the same into $HOME/sfw//libmesh-opt,
#   with PETSC_ARCH=linux-opt and make METHOD=opt (no cd line)
```

- “You need petsc 3.3-p5 for the most recent version of the Boyce branch.” IBFE, IBAMR’s finite element structures, needed libMesh, so spring 2013 rebuilt the stack around PETSc 3.3-p5 and added libMesh 0.8.0.
- libMesh 0.8.0 came from SourceForge’s Subversion and was built in its own source folder: `make METHOD=dbg` or `METHOD=opt`, no `make install`. IBAMR then pointed at that folder.
- Slips: no space between the SVN address and the target folder (`.../libmesh$HOME/sfw//libmesh-dbg`, with a double slash), so `svn` gets one argument; `/path/to/libmesh-dbg` is a placeholder, and the optimized slide has no `cd` at all.

The instructor’s MATH 769 slides “IBFE install” (April 11, 2013), slides 2–4; the host name in the title replaced.

SPRING 2013 (2/3): IBAMR WITH LIBMESH

MATH 769 “IBFE install” slides 5–6 • historical, do not run

```
 cd $HOME/sfw
 mkdir ibamr
 cd ibamr
svn co https://ibamr.googlecode.com/svn/branches/boyceg IBAMR
 cd $HOME/sfw/ibamr
 mkdir ibamr-objs-debug
 cd ibamr-objs-debug
 export PETSC_ARCH=linux
 export PETSC_DIR=$HOME/sfw/petsc/3.3-p5
 ../IBAMR/configure --config-cache \
   CFLAGS="-g -O1 -Wall" \
   CXXFLAGS="-g -O1 -Wall" \
   FFLAGS="-g -O1 -Wall" \
   FCFLAGS="-g -O1 -Wall" \
```

continued • historical, do not run

```
   CC=$HOME/sfw/linux/openmpi/1.4.3/bin/mpicc \
   CXX=$HOME/sfw/linux/openmpi/1.4.3/bin/mpicxx \
   F77=$HOME/sfw/linux/openmpi/1.4.3/bin/mpif90 \
   FC=$HOME/sfw/linux/openmpi/1.4.3/bin/mpif90 \
   MPICC=$HOME/sfw/linux/openmpi/1.4.3/bin/mpicc \
   MPICXX=$HOME/sfw/linux/openmpi/1.4.3/bin/mpicxx \
   CPPFLAGS="-DOMPI_SKIP_MPICXX" \
   --with-hypre=$PETSC_DIR/$PETSC_ARCH \
   --with-samrai=$HOME/sfw/samrai/2.4.4/linux-g++-debug \
   --with-hdf5=$HOME/sfw/linux/hdf5/1.8.7 \
   --with-blitz=$HOME/sfw/linux/blitz/0.9 \
   --with-silo=$HOME/sfw/linux/silo/4.8 \
     --enable-libmesh \
     --with-libmesh=$HOME/sfw/libmesh-dbg \
     --with-libmesh-compile-mode=dbg
make lib
make check
```

The instructor’s MATH 769 slides “IBFE install” (April 11, 2013), slides 5–6.

SPRING 2013 (3/3): WHAT THE LIBMESH OPTIONS DO, THE OPTIMIZED BUILD, AND HDF5

- The 2012 configure plus three options: `--enable-libmesh`, `--with-libmesh` (the libMesh source folder) and `--with-libmesh-compile-mode=dbg` (which of libMesh's builds to use). `make lib` builds the libraries only.
- Optimized (slide 7): `ibamr-objs-opt`, `PETSC_ARCH=linux-opt`, `"-O3 -Wall"`, `linux-g++-opt`, `--with-libmesh=$HOME/sfw/libmesh-opt`, `--with-libmesh-compile-mode=opt`; then `make` and `make check`.
- Slide 8, HDF5 on the lab servers: "NEW TO `<host>`1/2: *Need to use `CFLAGS=-O1 ./configure \` to get it to work on a newer compiler on ubuntu!"
- The debug PETSc build is still called `linux`.

The instructor's MATH 769 slides "IBFE install" (April 11, 2013), slides 5–8. Slides 9–13 (FE output in VisIt; the mesh formats libMesh reads) belong to chapters 28 and 17–18.

LETTING PETSC BUILD OPENMPI (THLINUX, APRIL 2013)

"THLinuxInstall" (MATH 769), debug • historical, do not run

```
PETSC debug and opt (with mpi)
export PETSC_DIR=$PWD
export PETSC_ARCH=linux
./config/configure.py \
CC=gcc \
CXX=g++ \
FC=gfortran \
F77=gfortran \
--download-openmpi=1 \
--with-openmpi=1 \
--disable-mpi-cxx-seek \
--disable-heterogeneous \
--enable-orterun-prefix-by-default \
--with-default-arch=0 \
--PETSC_ARCH=$PETSC_ARCH \
--with-debugging=1 \
--with-c++-support \
--with-hypre=1 \
--download-hypre=1
```

optimized • historical, do not run

```
export PETSC_DIR=$PWD
export PETSC_ARCH=linux-opt
./config/configure.py \
--CC=gcc \
--CXX=g++ \
--FC=gfortran \
--F77=gfortran \
--COPTFLAGS="-O3" \
--CXXOPTFLAGS="-O3" \
--FOPTFLAGS="-O3" \
--download-openmpi \
--with-openmpi=1 \
--disable-mpi-cxx-seek \
--disable-heterogeneous \
--enable-orterun-prefix-by-default \
--with-default-arch=0 \
--PETSC_ARCH=$PETSC_ARCH \
--with-debugging=0 \
--with-c++-support \
--download-f-blas-lapack=yes \
--download-f-blas-lapack=1 \
--with-hypre=1 \
--download-hypre=1 \
--with-x=0
```

- `--download-openmpi`: PETSc builds its own OpenMPI, and SAMRAI then uses PETSc's `mpicc`: `--with-MPICC=$HOME/sfw/petsc/3.3-p5/linux/bin/mpicc`.
- SAMRAI with patch 121212; libMesh 0.8.0 as on slide 33 (with a space before the target folder, and a real `cd` for the debug build).
- Slips: the debug line gives the compilers as `CC=gcc` (no `--`), the optimized one as `--CC=gcc`; three of OpenMPI's own configure options are handed to PETSc's configure; no `make` or `make test` for PETSc; the optimized libMesh is configured without `cd $HOME/sfw/libmesh-opt`.

The instructor's MATH 769 course folder, "THLinuxInstall" (modified April 30, 2013).

KILLDEVIL HOME INSTALL (2012–13) (1/5): SHELL, MODULE, DISK

slide 1: “(1) add these lines to .bash_profile” • historical, do not run

```
export FC=gfortran CC=gcc CXX=g++ F77=gfortran
# It looks like:
if [ -f /etc/profile.modules ]
then
        . /etc/profile.modules
        export FC=gfortran
        export CC=gcc
        export CXX=g++
        export F77=gfortran
fi
# (2) in the terminal type:
module initadd openmpi_gcc/4.4.6
```

slide 3: “IBAMR install” • historical, do not run

```
Log in as root
cd $HOME
mkdir sfw2
cd sfw2
Move all installation files here
mkdir linux
cd linux
```

- The compilers are set once in `.bash_profile`, inside the block that loads the cluster’s modules; `module initadd` makes the GNU OpenMPI module load at every login.
- Slide 2, disk space: “Warning! For a second install with libmesh you will not have sufficient space. You can either submit a ticket for more space on killdevil, or delete your old install”.
- “Log in as root” is copied from the THLinux slides: on a cluster you have no root, and the install goes into `$HOME/sfw2` anyway.
- Then Blitz++, HDF5, Silo and OpenMPI 1.4.3 as in 2012 (slides 13–14), in `$HOME/sfw2/linux`.

The instructor’s “Install IBAMR on KillDevil” slides 1–7 (about 2012–13).

KILLDEVIL HOME INSTALL (2/5): PETSC 3.3-P5

slides 8–9 • historical, do not run

```
cd $HOME/sfw2
mkdir petsc
cd petsc
tar xvfz petsc-3.3-p5.tar.gz
cd petsc-3.3-p5
export PATH=$HOME/sfw2/linux/openmpi/1.4.3/bin:$PATH
export MANPATH=$HOME/sfw2/linux/openmpi/1.4.3/share/man:$MANPATH
export LD_LIBRARY_PATH="$HOME/sfw2/linux/openmpi/1.4.3/lib"
# debug
export PETSC_DIR=$PWD
export PETSC_ARCH=linux
./config/configure.py \
--CC=$HOME/sfw2/linux/openmpi/1.4.3/bin/mpicc \
--CXX=$HOME/sfw2/linux/openmpi/1.4.3/bin/mpicxx \
--FC=$HOME/sfw2/linux/openmpi/1.4.3/bin/mpif90 \
--LDFLAGS="-L$HOME/sfw2/linux/openmpi/1.4.3/lib -Wl,-rpath,$HOME/sfw2/
      ↪ linux/openmpi/1.4.3/lib" \
--with-default-arch=0 \
--PETSC_ARCH=$PETSC_ARCH \
--with-debugging=1 \
--with-c++-support \
--with-hypre=1 \
--download-hypre=1
make
make test
```

- The home-built OpenMPI is put first on `PATH` and `MANPATH`, and `LD_LIBRARY_PATH` is set to its `lib` folder, before PETSc is configured.
- The slide's own note: "I used to add this line at the beginning `export LD_LIBRARY_PATH=$HOME/sfw2/linux/openmpi/1.4.3/lib` I got a gcc error since it removed gcc-4.4.3 from my library path." Setting the variable to one folder drops everything the modules had put there, yet slide 8 itself still does so (with quotes); prepending (`...:$LD_LIBRARY_PATH`) keeps the modules' entries.
- Optimized (slide 10): `PETSC_ARCH=linux-opt`; adds `--COPTFLAGS="-O3" --CXXOPTFLAGS="-O3" --FOPTFLAGS="-O3"`, `--with-debugging=0`, `--download-f-blas-lapack=yes`, `--download-f-blas-lapack=1`, `--with-x=0`; `make`, `make test`.

The instructor's "Install IBAMR on KillDevil" slides 8–10.

KILLDEVIL HOME INSTALL (3/5): SAMRAI AND LIBMESH 0.8.0

slide 11 • historical, do not run

```
cd $HOME/sfw2
mkdir samrai
cd samrai
mkdir 2.4.4
cd 2.4.4
tar xvfz SAMRAI-v2.4.4.tar.gz
cd SAMRAI
./source/scripts/includes --link
gunzip -c SAMRAI-v2.4.4-patch-121212.gz | patch -p2
```

Slides 12–13: SAMRAI's debug and optimized configures (slide 18), after `export PETSC_ARCH=linux` or `linux-opt`.

A slip: the patch command runs inside `SAMRAI`, but the `.gz` sits one folder up, beside the SAMRAI tarball; it needs `../SAMRAI-v2.4.4-patch-121212.gz`, as in 2012.

slide 14 (15: the same for `libmesh-opt`, `linux-opt`, `METHOD=opt`) • historical, do not run

```
svn co  https://libmesh.svn.sourceforge.net/svnroot/libmesh/
      ↪ tags/libmesh-0.8.0/libmesh $HOME/sfw2/libmesh/
      ↪ libmesh-dbg
cd $HOME/sfw2/libmesh/libmesh-dbg
./configure \
PETSC_DIR=$HOME/sfw2/petsc/petsc-3.3-p5 \
PETSC_ARCH=linux \
CC=gcc \
CXX=g++ \
FC=gfortran \
F77=gfortran \
--enable-everything --disable-shared --disable-tbb --disable-
      ↪ cppthreads --disable-exceptions
make METHOD=dbg
http://www.mail-archive.com/libmesh-devel@lists.sourceforge.
      ↪ net/msg04710.html
```

Two options more than in April 2013, `--disable-tbb` and `--disable-cppthreads`, followed on the slide by a link to a message on the libMesh developers' mailing list.

The instructor's "Install IBAMR on KillDevil" slides 11–15.

KILLDEVIL HOME INSTALL (4/5): IBAMR IBAMR-DEV AT REVISION 3606

slides 16–17 • historical, do not run

```
cd $HOME/sfw2
mkdir ibamr
cd ibamr
svn co https://ibamr.googlecode.com/svn/branches/ibamr-dev
      ↪ IBAMR2
# "The instructions in this powerpoint only work for an older
#  version of IBAMR. So now you want to go to an older revision
      ↪ :"
cd IBAMR2
svn up -r3606
cd $HOME/sfw2/ibamr
mkdir ibamr-objs-debug
cd ibamr-objs-debug
export PETSC_ARCH=linux
export PETSC_DIR=$HOME/sfw2/petsc/petsc-3.3-p5
../IBAMR2/configure --config-cache \
CFLAGS="-g -O1 -Wall" \
CXXFLAGS="-g -O1 -Wall" \
FFLAGS="-g -O1 -Wall" \
FCFLAGS="-g -O1 -Wall" \
```

continued • historical, do not run

```
CC=$HOME/sfw2/linux/openmpi/1.4.3/bin/mpicc \
CXX=$HOME/sfw2/linux/openmpi/1.4.3/bin/mpicxx \
F77=$HOME/sfw2/linux/openmpi/1.4.3/bin/mpif90 \
FC=$HOME/sfw2/linux/openmpi/1.4.3/bin/mpif90 \
MPICC=$HOME/sfw2/linux/openmpi/1.4.3/bin/mpicc \
MPICXX=$HOME/sfw2/linux/openmpi/1.4.3/bin/mpicxx \
CPPFLAGS="-DOMPI_SKIP_MPICXX" \
--with-hypre=$PETSC_DIR/$PETSC_ARCH \
--with-samrai=$HOME/sfw2/samrai/2.4.4/linux-g++-debug \
--with-hdf5=$HOME/sfw2/linux/hdf5/1.8.7 \
--with-blitz=$HOME/sfw2/linux/blitz/0.9 \
--with-silo=$HOME/sfw2/linux/silo/4.8 \
--enable-libmesh \
--with-libmesh=$HOME/sfw2/libmesh/libmesh-dbg \
--with-libmesh-compile-mode=dbg
make lib
make check
```

The instructor's "Install IBAMR on KillDevil" slides 16–17.

KILLDEVIL HOME INSTALL (5/5): THE OPTIMIZED BUILD AND THE FIRST RUN

- The `ibamr-dev` branch, pinned to revision 3606 because the slides were written for it; the source folder is `IBAMR2`.
- Optimized (slide 18): `ibamr-objs-opt`, `linux-opt`, `"-O3 -Wall"`, `linux-g++-opt`, `libmesh-opt`, `--with-libmesh-compile-mode=opt`; `make`, `make check`.
- Run (slide 19), with a slip: `cd $HOME/sfw/ibamr/ibamr-objs-debug/...` names `sfw`, not `sfw2`.

slide 19 • historical, do not run

```
cd $HOME/sfw/ibamr/ibamr-objs-debug/examples/IB/
    ↪ staggered_grid/explicit/ex0
make examples
./main2d input2d
```

The instructor's "Install IBAMR on KillDevil" slides 18–19.

KILLDEVIL: “WORKING DIRECTIONS FOR NEWER IBAMR -- IN PROGRESS”

“Install IBAMR on KillDevil”, slide 20 • historical, do not run

```
cd $HOME/sfw2/ibamr
mkdir ibamr-objs-debug2
cd ibamr-objs-debug2
export LD_LIBRARY_PATH=$HOME/sfw2/linux/openmpi/1.4.3/lib
export PETSC_ARCH=linux
export PETSC_DIR=$HOME/sfw2/petsc/petsc-3.3-p5
sh ../IBAMR/configure --config-cache \
# (here: the debug flags and MPI wrappers of slide 17)
--with-boost=/sw \
--with-hypre=$PETSC_DIR/$PETSC_ARCH \
--with-samrai=$HOME/sfw2/samrai/2.4.4/linux-g++-debug \
--with-hdf5=$HOME/sfw2/linux/hdf5/1.8.7 \
--with-silo=$HOME/sfw2/linux/silo/4.8 \

make
make check
```

An unfinished attempt at the post-December-2013 IBAMR (no Blitz++, Boost instead). Its problems:

- `--with-boost=/sw` is the Fink folder of the developer’s Mac (slide 47); on KillDevil there is no `/sw`;
- it configures `../IBAMR`, but the checkout is `IBAMR2`;
- it leaves out libMesh;
- its last option ends with a backslash, made harmless by the empty line after it (without that line, `make` would become an argument of configure);
- `export LD_LIBRARY_PATH=...` replaces the whole variable, the gcc problem of slide 38.

Slides 21–22 (running on KillDevil, changing spring parameters) belong to chapters X1 and X3.

The instructor’s “Install IBAMR on KillDevil” slide 20.

THLINUX, AS ROOT (2013) (1/2): WHAT DIFFERS FROM KILLDEVIL

The THLinux slides repeat the KillDevil recipe with these differences:

- “IBAMR install / Log in as root / cd /usr/bin / mkdir sfw / cd sfw / Move all installation files here / mkdir linux / cd linux”: the stack in `/usr/bin/sfw`;
- `sh ./configure` for Blitz++, Silo, OpenMPI and libMesh; the Blitz prefix `/usr/bin/sfw/linux/blitz-0.9\` (no space);
- the PETSc debug slide starts with `export LD_LIBRARY_PATH=/usr/bin/sfw/linux/openmpi/1.4.3/lib`;
- libMesh 0.8.0 checked out to `/usr/bin/sfw/libmesh/libmesh-dbg`, then `cd libmesh-dbg` (relative) and `sh ./configure PETSC_DIR=/usr/bin/sfw/petsc/petsc-3.3-p5 ...`;
- IBAMR from `branches/boyceg` (no revision pin) into `IBAMR2`; `sh ../IBAMR2/configure --config-cache`;
- the text names patch 111217, the command applies 121212, from the wrong folder (slide 39).

The instructor’s “Install THLinux and IBAMR” slides 5–20 (2013; the deck’s Dropbox note dates from about 2011).

THLINUX, AS ROOT (2/2): THE LD_LIBRARY_PATH RITUAL, AND MAKEFILE PATHS

THLinux slide 21 = “Running IBAMR on THLinux” slide 2 • historical, do not run

```
Add #include <ibtk/LDataManager.h>  to main.C
IBAMR_SRC_DIR = /usr/bin/sfw/ibamr/IBAMR2
IBAMR_BUILD_DIR = /usr/bin/sfw/ibamr/ibamr-objs-opt
# "You must type this in your terminal every time
#  you run main2d/3d:"
export LD_LIBRARY_PATH=/usr/bin/sfw/linux/openmpi/1.4.3/lib
# "Adding this to .bash_profile and .bashrc does not
#  seem to fix it."
```

The include was the fix for one compile error of that IBAMR (chapter X3); the `export` had to be typed at every login because the home-built OpenMPI’s libraries were not on the system’s library path.

“Running IBAMR on KillDevil”, slide 2 • historical, do not run

```
IBAMR_SRC_DIR = $HOME/sfw2/ibamr/IBAMR2
IBAMR_BUILD_DIR = $HOME/sfw2/ibamr/ibamr-objs-opt
# "Warning - Something changed on killdevil so my home
      ↪ directory
#  was messed up, and I had to put in the full path."
IBAMR_SRC_DIR = /pathto/sfw2/ibamr/IBAMR2
IBAMR_BUILD_DIR = /pathto/sfw2/ibamr/ibamr-objs-opt
```

The same Makefile lines on KillDevil, first with `$HOME`, then with the full path. In a Makefile `$HOME` reads as `$H` followed by `OME`: `$H` is empty, so the path became `OME/sfw2/....` `$(HOME)` would have worked, then as now (chapter X1).

The instructor’s “Install THLinux and IBAMR” slide 21, “Running IBAMR on THLinux” and “Running IBAMR on KillDevil” slides (2013–15; the home path is shown as `/pathto`). Running, screen and etiquette: chapter X1.

DECEMBER 2013 (1/2): BLITZ++ OUT, BOOST AND EIGEN IN

Boyce Griffith's advice for the December 2013 rebuild (e-mailed, kept in the instructor's working notes; summarized):

- third-party libraries for the current code: PETSc 3.3, HDF5 1.8.x, a recent Silo (he used 4.9.1), SAMRAI 2.4.4 with the patches from IBAMR's web site;
- just committed to `branches/ibamr-dev`: Blitz++ removed. Boost's MultiArray replaces Blitz's Array class and Boost's Array replaces `TinyVector` (bundled in `ibtk/contrib/boost`, or `--with-boost=PATH`);
- new NDIM-dimensional `Matrix`, `Vector` and `Point` classes based on Eigen (Vector and Point interchangeable, Point by convention for positions; bundled in `ibtk/contrib/eigen`, or `--with-eigen=PATH`);

> This is the line between the two eras of course code: before it, `TinyVector<double,NDIM>` positions; after it, `Point` (chapter X3, slide "Target points: four types for one position").

The instructor's "December 2013 IBAMR install working notes", slides 2–4 (Boyce Griffith's e-mailed notes, summarized).

DECEMBER 2013 (2/2): MUPARSER, AN INSTALLED LIBMESH, AND WHAT CONFIGURE READS FROM PETSC

- muParser moved to `ibtk/contrib/muparser` and updated to 2.2.3 (or `--with-muparser=PATH`);
- libMesh 0.9.1 supported, which needs an installed libMesh (configure, make, make install), pointed at with `--with-libmesh=DIR`; and `--with-libmesh-compile-mode=METHOD` renamed `--with-libmesh-method=METHOD`;
- with libMesh, Boost must be installed (unpacked is enough: header-only) and the same Boost given to libMesh and IBAMR;
- configure now reads the hypre, HDF5, BLAS and LAPACK information from PETSc.

The same notes (slides 5–22) then show a configure line from the developer's Mac and the instructor's THLinux rebuild: the next four slides.

The instructor's "December 2013 IBAMR install working notes", slides 2–4 (Boyce Griffith's e-mailed notes, summarized).

A CONFIGURE LINE FROM THE DEVELOPER'S MAC (DECEMBER 2013) (1/2)

working notes, slide 5 (the prompt scrubbed) • historical, do not run

```
<host>:ibamr-objs-dbg <user>$ cat configure_command
export PETSC_ARCH=darwin
export PETSC_DIR=$HOME/sfw/petsc/3.3-p5
../IBAMR-dev/configure --config-cache \
 CFLAGS="-g -W -Wall -Wextra" \
 CXXFLAGS="-g -W -Wall -Wextra" \
 FCFLAGS="-g -W -Wall -Wextra" \
 CC=$PETSC_DIR/$PETSC_ARCH/bin/mpicc \
 CXX=$PETSC_DIR/$PETSC_ARCH/bin/mpicxx \
 FC=gfortran \
 MPICC=$PETSC_DIR/$PETSC_ARCH/bin/mpicc \
 MPICXX=$PETSC_DIR/$PETSC_ARCH/bin/mpicxx \
 --with-boost=/sw \
 --with-hdf5=/sw \
 --with-samrai=$HOME/sfw/samrai/2.4.4/darwin-g++-dbg \
 --with-silo=$HOME/sfw/darwin/silo/4.8 \
 --enable-libmesh \
 --with-libmesh=$HOME/sfw/darwin/libmesh/0.9.1-dbg \
 --with-libmesh-method=dbg
```

`/sw` is Fink's folder on a Mac: Boost and HDF5 from the package manager. The compilers are the MPI wrappers PETSc built (`$PETSC_DIR/$PETSC_ARCH/bin/mpicc`), and the new option name `--with-libmesh-method` is used.

The instructor's December 2013 working notes, slide 5 (Boyce Griffith's example; machine and user name replaced).

A CONFIGURE LINE FROM THE DEVELOPER'S MAC (2/2): THE DEBUG AND OPTIMIZED VARIANTS

- Slide 6, debug ("I have had PETSc download/install hypre for me, but I am using a system installation of HDF5"): `../IBAMR/configure --config-cache --enable-maintainer-mode`, `CFLAGS="-g -O0 -W -Wall -Wextra"`, `CXXFLAGS="-g -O0 -W -Wall -Wextra -Wno-variadic-macros"`, `FCFLAGS="-g -O0 -W -Wall -Wextra"`, the same wrappers and options, `darwin-g++-debug`.
- Slide 7, optimized: `PETSC_ARCH=darwin-opt`, `-O3 -W -Wall -Wextra` (`-Wno-variadic-macros` in `CXXFLAGS`), `darwin-g++-opt`, `0.9.1-opt`, `--with-libmesh-method=opt`.

`-Wno-variadic-macros` silences a warning about macros with a variable number of arguments; `--enable-maintainer-mode` is an autotools option for developers who change IBAMR's build files.

The instructor's December 2013 working notes, slides 6–7.

THE THLINUX REBUILD OF DECEMBER 2013 (1/2): LIBMESH 0.9.2 FROM GITHUB

working notes, slides 18–19 • historical, do not run

```
https://github.com/libMesh/libmesh/releases
/usr/bin/sfw/libmesh/libmesh-dbg
move to /usr/bin/sfw/libmesh, unzip, and rename libmesh-dbg
cd libmesh-dbg
sh ./configure \
PETSC_DIR=/usr/bin/sfw/petsc/petsc-3.3-p5 \
PETSC_ARCH=linux \
CC=gcc \
CXX=g++ \
FC=gfortran \
F77=gfortran \
--enable-everything --disable-shared --disable-tbb --disable-
      ↪ cppthreads --disable-exceptions
make METHOD=dbg
make
make install
# slide 18 then ends with the mailing-list link of the KillDevil
      ↪ slides and "Libmesh-0.9.2"
# optimized slide: the same text "rename libmesh-dbg", then cd
      ↪ libmesh-opt,
#   PETSC_ARCH=linux-opt, make METHOD=opt; make; make install
```

- The stack as in the THLinux root install (`/usr/bin/sfw`) but without Blitz++: HDF5 1.8.7, Silo 4.8, OpenMPI 1.4.3, PETSc 3.3-p5 (debug and optimized), SAMRAI 2.4.4 with 121212, now with the fixed patch command `gunzip -c ../SAMRAI-v2.4.4-patch-121212.gz | patch -p2`.
- libMesh 0.9.2, downloaded from libMesh's GitHub releases, and now `make install`ed, as the new IBAMR requires.
- But the configure has no `--prefix`, so `make install` puts libMesh in autoconf's default, `/usr/local`, while IBAMR is still pointed at the source folder (`--with-libmesh=/usr/bin/sfw/libmesh/libmesh-dbg`).
- The optimized slide repeats "rename libmesh-dbg".

The instructor's December 2013 working notes, slides 8–19.

THE THLINUX REBUILD OF DECEMBER 2013 (2/2): IBAMR IBAMR-DEV

working notes, slides 20–22 • historical, do not run

```
cd /usr/bin/sfw
mkdir ibamr
cd ibamr
svn co https://ibamr.googlecode.com/svn/branches/ibamr-dev
      ↪ IBAMR2
# optimized configure (slide 22) ends:
--with-boost=/sw \
--with-hypre=$PETSC_DIR/$PETSC_ARCH \
--with-samrai=/usr/bin/sfw/samrai/2.4.4/linux-g++-opt \
--with-hdf5=/usr/bin/sfw/linux/hdf5/1.8.7 \
--with-silo=/usr/bin/sfw/linux/silo/4.8
make
make check
```

slide 23, “Specific TH Linux reminders” • historical, do not run

```
export LD_LIBRARY_PATH=/usr/bin/sfw/linux/openmpi/1.4.3/lib
```

Code that broke with it (slide 24 of the notes; chapter X3): the `*_prefix_config.h` includes had to be commented out, and target positions changed from `TinyVector<double,NDIM>&` to `Point&`.

The debug configure (slide 21) is the THLinux debug configure with `export LD_LIBRARY_PATH=/usr/bin/sfw/linux/openmpi/1.4.3/lib` first, `../IBAMR2/configure` (no `sh`), no `--with-blitz`, `--with-boost=/sw`, and still `--with-libmesh=/usr/bin/sfw/libmesh/libmesh-dbg` `--with-libmesh-compile-mode=dbg`.

What the December notes got wrong

- the debug configure keeps the old option name `--with-libmesh-compile-mode`, although slide 4 says it was renamed `--with-libmesh-method`;
- the optimized configure has no libMesh at all;
- `--with-boost=/sw`, the Fink folder of the developer’s Mac, is copied onto Linux;
- libMesh is installed without `--prefix` (last slide);
- the SVN addresses are dead.

A separate note of the same month, “Notes on THLinux”, gave THLinux a VirtualBox recipe (slide 11).

The instructor’s December 2013 working notes, slides 20–24.

2014: THE KILLDEVIL MODULE IBAMR/3803

"IBAMR Killdevil Module" (March 26, 2014), slides 2–4 • historical, do not run

```
# slide 2, "Load the ibamr module":
# "You must first remove any other module."
module remove openmpi___
# "You may also need to delete manually in your .profile, .bashrc,
      ↪
#  and .bash_profile files in your home directory."
module add ibamr/3803
# In Makefile
IBAMR_SRC_DIR = /nas02/apps/ibamr-3803/OFED-1.5.2/IBAMR2
IBAMR_BUILD_DIR = /nas02/apps/ibamr-3803/ibamr-objs-opt
# "Path to ibamr files in general": /nas02/apps/ibamr-3803
```

"Using IBAMR Module on Kure" (2014), after `module initadd ibamr/3803` and a new login • historical, do not run

```
[<user>@<host> ~]$ module list
1) null 2) openmpi_intel/11.1 3) hdf5/1.8.5 4) petsc/3.4 5)
      ↪ libmesh/0.9.3 6) silo/4.9.1 7) boost/1.53.0 8) ibamr
      ↪ /3803
```

- From 2014 nobody had to build the stack on KillDevil: UNC's IT services (ITS) installed IBAMR as a module, and a program's Makefile pointed at it.
- The module list is the version matrix of 2014: the Intel 11.1 build of OpenMPI, HDF5 1.8.5, PETSc 3.4, libMesh 0.9.3, Silo 4.9.1, Boost 1.53.0, and IBAMR at revision 3803 (the module was installed about February 2014, "Feb. 2014 (?)" in the course Makefiles; before IBAMR's first tagged release, v0.1-rc1 of May 2014).
- The course examples said so: "These examples use IBAMR version 3803. I only update the examples once every 6-12 months when we update IBAMR on the UNC cluster."
- `openmpi___` is a placeholder for whichever OpenMPI module was loaded. Loading modules, the queues and the rest of the cluster work are in chapter X1; the code changes this module brought are in chapter X3.

The instructor's "IBAMR Killdevil Module" slides 2–4 (2014); the undergraduate guide "Using IBAMR Module on Kure" (2014; author not recorded; prompt scrubbed); the course examples' READ ME.

2014: THE INTEL RUNTIME ERROR, AND ITS FIX

"IBAMR Killdevil Module", slide 9, "LD_LIBRARY_PATH" • historical, do not run

```
# "If you get this:"
/usr/mpi/intel/openmpi-1.4.1/bin/orted: error while loading shared
      ↪ libraries: libimf.so: cannot open shared object file: No such
      ↪ file or directory
# "Then try this:"
export LD_LIBRARY_PATH=/nas02/apps/intel-11.1/cc/lib/intel64/
```

- The module's OpenMPI was built with Intel's compilers; its launcher daemon `orted` needs Intel's math library `libimf.so` at run time.
- The fix puts the Intel 11.1 libraries on the library path. Like the 2013 `export` lines, it replaces the variable rather than adding to it.
- The same kind of error (a shared library not found) is what the `-rpath` in the PETSc configures of 2012–19 was there to prevent.

The instructor's "IBAMR Killdevil Module" slide 9 (2014). Slides 5–8 (code changes) are in chapter X3.

2015: A NEWER IBAMR AGAINST THE MODULE'S LIBRARIES (ALEX HOOVER) (1/2)

"Why do this?": "Sometimes it's necessary to use updated versions of IBAMR and asking ITS to update the module might take too long/too much work" and "We want to use the module directories of the libraries that Killdevil uses. This avoids the having to install SAMRAI, PETSc, Libmesh, etc. . . "

"Installing IBAMR on Killdevil using Modules" (2015) • historical, do not run

```
Log on to killdevil.unc.edu
cd path/to/ibamr
module load ibamr         (this is done for the modules)
module load git
git clone https://github.com/IBAMR/IBAMR.git
cd IBAMR
git checkout 386ab2
cd ..
mkdir ibamr-objs-opt
cd ibamr-objs-opt
../IBAMR/configure --config-cache CFLAGS="-g -O3 -Wall" CXXFLAGS="-g -O3  -Wall" FFLAGS="-g -O3 -Wall" FCFLAGS="-g -O3 -Wall" CC=
      ↪ mpicc CXX=mpicxx F77=mpif90 FC=mpif90 MPICC=mpicc MPICXX=mpicxx --with-hypre=/nas02/apps/petsc-3.4/arch-linux2-c-opt --with
      ↪ -samrai=/nas02/apps/ibamr-3803/samrai-2.4.4/linux-opt --with-hdf5=/nas02/apps/hdf5-1.8.5 --with-boost=/nas02/apps/boost
      ↪ -1.53.0 --with-silo=/nas02/apps/silo-4.9.1 --enable-libmesh --with-libmesh=/nas02/apps/libmesh-0.9.3
make
make check
```

Alex Hoover, "Installing IBAMR on Killdevil using Modules" (2015), slides 1–3.

2015: A NEWER IBAMR AGAINST THE MODULE'S LIBRARIES (2/2)

slide 4: "Remember to change your Makefiles!" • historical, do not run

```
IBAMR_SRC_DIR = $HOME/path/to/ibamr/IBAMR
IBAMR_BUILD_DIR = $HOME/path/to/ibamr/ibamr-objs-opt
```

- The first recipe in this chapter that gets IBAMR from GitHub with `git`, pinned to commit `386ab2`.
- Only IBAMR is built; PETSc 3.4 (`arch-linux2-c-opt`), SAMRAI 2.4.4 (from the `ibamr-3803` module), HDF5 1.8.5, Boost 1.53.0, Silo 4.9.1 and libMesh 0.9.3 are the module's.

What the recipe leaves to the modules, and its gaps

- no `PETSC_DIR` or `PETSC_ARCH`, which the loaded modules are left to supply, and no `--with-libmesh-method`, whose default, `opt`, matches the module's optimized libMesh;
- no `CPPFLAGS="-DOMPI_SKIP_MPICXX"`;
- `-g -O3`: debug symbols in the optimized build (larger files, same speed);
- a pinned 2015 commit: newer IBAMR may need newer libraries than the module's;
- `path/to/ibamr`: a placeholder.

autoibamr does the opposite: it builds IBAMR's libraries itself (SAMRAI, PETSc, libMesh, HDF5, Silo, Boost) on the machine's compilers and MPI, so that one script works on any machine (chapter 2).

Alex Hoover, "Installing IBAMR on Killdevil using Modules" (2015), slide 4; chapter 2.

OS X MAVERICKS (ABOUT 2015) (1/6): MACPORTS GCC 4.9 AND HDF5

OS X Mavericks 10.9.5 notes (unattributed) • historical, do not run

```
mkdir $HOME/Softwares
sudo port install gcc49
mkdir $HOME/Softwares/HDF5
cd $HOME/Softwares/HDF5
wget http://www.hdfgroup.org/ftp/HDF5/current/src/hdf5-1.8.15-
      ↪ patch1.tar
tar xvf hdf5-1.8.15-patch1.tar
mv hdf5-1.8.15-patch1 1.8.15
mv hdf5-1.8.15-patch1.tar 1.8.15/
cd 1.8.15
./configure \
  CC=gcc-mp-4.9 \
  CXX=g++-mp-4.9 \
  FC=gfortran-mp-4.9 \
  F77=gfortran-mp-4.9 \
  --enable-production \
  --disable-debug \
  --prefix=$HOME/Softwares/HDF5/1.8.15
make
make check
make install
```

the same notes, Silo and OpenMPI (same pattern; summarized) • historical, do not run

```
# Silo:
wget https://wci.llnl.gov/content/assets/docs/simulation/
      ↪ computer-codes/silo/silo-4.10.2/silo-4.10.2.tar.gz
#   --prefix=$HOME/Softwares/Silo/4.10.2 --disable-silex
#   make; make check; make install
# OpenMPI:
wget http://www.open-mpi.org/software/ompi/v1.10/downloads/
      ↪ openmpi-1.10.0.tar.gz
#   --prefix=$HOME/Softwares/OpenMPI/1.10.0
#   --disable-mpi-cxx-seek --disable-heterogeneous
#   --enable-orterun-prefix-by-default
```

- A Mac build with MacPorts' gcc 4.9 (`gcc-mp-4.9`, `g++-mp-4.9`, `gfortran-mp-4.9`), every package in `$HOME/Softwares/<Package>/<version>`, each download fetched with `wget`.
- PETSc, SAMRAI, libMesh and IBAMR are built in debug form only.

OS X Mavericks 10.9.5 install notes (about 2015; kept in the course's old install notes; unattributed).

OS X MAVERICKS (2/6): PETSC 3.5.4 AND SAMRAI

Mavericks notes, PETSc • historical, do not run

```
mkdir $HOME/Softwares/PETSc
cd $HOME/Softwares/PETSc
wget http://ftp.mcs.anl.gov/pub/petsc/release-snapshots/petsc
      ↪ -3.5.4.tar.gz
tar xvfz petsc-3.5.4.tar.gz
mv petsc-3.5.4 3.5.4/
cd 3.5.4
mkdir linux-dbg
export PETSC_DIR=$HOME/Softwares/PETSc/3.5.4
export PETSC_ARCH=linux-dbg
./config/configure.py \
  --CC=$HOME/Softwares/OpenMPI/1.10.0/bin/mpicc \
  --CXX=$HOME/Softwares/OpenMPI/1.10.0/bin/mpicxx \
  --FC=$HOME/Softwares/OpenMPI/1.10.0/bin/mpif90 \
  --LDFLAGS="-L$HOME/Softwares/OpenMPI/1.10.0/lib -Wl,-rpath,
      ↪ $HOME/Softwares/OpenMPI/1.10.0/lib" \
  --PETSC_ARCH=linux-dbg \
  --with-shared-libraries \
  --download-hypre
make
make test
```

Mavericks notes, SAMRAI • historical, do not run

```
mkdir $HOME/Softwares/SAMRAI
cd $HOME/Softwares/SAMRAI
wget SAMRAI
wget https://github.com/IBAMR/IBAMR/releases/download/v0.1-
      ↪ rc1/SAMRAI-v2.4.4-patch-121212.gz
tar xvfz SAMRAI-v2.4.4.tar.gz
mv SAMRAI 2.4.4
mv *.gz 2.4.4/
cd 2.4.4
./source/scripts/includes --link
gunzip -c $HOME/Softwares/SAMRAI/2.4.4/SAMRAI-v2.4.4-patch
      ↪ -121212.gz | patch -p2
mkdir linux-dbg
cd linux-dbg
# ../configure \  with the debug option list of slide 17,
#   prefix $HOME/Softwares/SAMRAI/2.4.4/linux-dbg
make
make install
```

`wget SAMRAI` is a placeholder: SAMRAI 2.4.4 had to be fetched by hand from LLNL. The patch, by now, came from IBAMR's GitHub release.

OS X Mavericks 10.9.5 install notes (about 2015; unattributed).

OS X MAVERICKS (3/6): BOOST 1.59.0, EIGEN 3.2.6 AND GSL 1.16

Mavericks notes, Boost and Eigen (unpacked, not built) • historical, do not run

```
mkdir $HOME/Softwares/Boost
cd $HOME/Softwares/Boost
wget http://sourceforge.net/projects/boost/files/latest/
      ↪ download?source=files
tar xvjf boost_1_59_0.tar.bz2
mv boost_1_59_0 1.59.0
cd 1.59.0
mkdir include
mv boost include/
mkdir $HOME/Softwares/Eigen
cd $HOME/Softwares/Eigen
wget http://bitbucket.org/eigen/eigen/get/3.2.6.tar.gz
tar xvfz 3.2.6.tar.gz
mv 3.2.6.tar.gz 3.2.6/
cd 3.2.6
mkdir include
mv eigen include/
```

Mavericks notes, GSL • historical, do not run

```
mkdir $HOME/Softwares/GSL
wget ftp://ftp.gnu.org/gnu/gsl/gsl-1.16.tar.gz
tar xvfz gsl-1.16.tar.gz
mv gsl-1.16 1.16
mv gsl-1.16.tar.gz  1.16/
cd 1.16
./configure \
  --prefix=$HOME/Softwares/GSL/1.16 \
  --disable-static \
  --disable-shared \
  CC=gcc-mp-4.9 \
  CXX=g++-mp-4.9 \
  F77=gfortran-mp-4.9 \
  FC=gfortran-mp-4.9
make
make install
```

Boost and Eigen are header-only for IBAMR: unpacked, with the headers moved under an `include` folder, as libMesh's `--with-boost` expected (IBAMR's configure also looks in the folder itself). GSL, the GNU Scientific Library, is built for IBAMR's optional `--enable-gsl`.

OS X Mavericks 10.9.5 install notes (about 2015; unattributed).

OS X MAVERICKS (4/6): LIBMESH 0.9.4

Mavericks notes, libMesh (each configure is one line in the notes) • historical, do not run

```
mkdir -p $HOME/Softwares/libMesh/0.9.4
wget https://github.com/libMesh/libmesh/releases/download/v0.9.4/libmesh-0.9.4.tar.gz
tar xvfz libmesh-0.9.4.tar.gz
mv libmesh-0.9.4 0.9.4
mv libmesh-0.9.4.tar.gz  0.9.4/
cd 0.9.4
mkdir linux-dbg
cd linux-dbg
../configure  --prefix=$HOME/Softwares/libMesh/0.9.4/linux-dbg  --with-methods=dbg  PETSC_DIR=$HOME/Softwares/PETSc/3.5.4
      ↪ PETSC_ARCH=linux-dbg  CC=$HOME/Softwares/OpenMPI/1.10.0/bin/mpicc  CXX=$HOME/Softwares/OpenMPI/1.10.0/bin/mpicxx  FC=$HOME/
      ↪ Softwares/OpenMPI/1.10.0/bin/mpif90  F77=$HOME/Softwares/OpenMPI/1.10.0/bin/mpif90  --with-boost=$HOME/Softwares/Boost
      ↪ /1.59.0  --with-eigen=$HOME/Softwares/Eigen/3.2.6  --enable-everything  --disable-libHilbert  --disable-cxx11
make
make install
```

libMesh is now configured in a build folder and installed, with `--prefix` and `--with-methods=dbg`, against the same Boost and Eigen that IBAMR gets next.

OS X Mavericks 10.9.5 install notes (about 2015; unattributed).

OS X MAVERICKS (5/6): IBAMR'S MASTER, WITH GSL

Mavericks notes, IBAMR • historical, do not run

```
mkdir -p $HOME/Softwares/IBAMR/v0
cd $HOME/Softwares/IBAMR/v0
git clone https://github.com/IBAMR/IBAMR.git
mkdir linux-dbg
cd linux-dbg
export PETSC_ARCH=linux-dbg
export PETSC_DIR=$HOME/Softwares/PETSc/3.5.4
../IBAMR/configure  CFLAGS="-O1 -Wall"  CXXFLAGS="-O1 -Wall"  FCFLAGS="-O1 -Wall"  CC=$HOME/Softwares/OpenMPI/1.10.0/bin/mpicc  CXX=
      ↪ $HOME/Softwares/OpenMPI/1.10.0/bin/mpicxx  FC=$HOME/Softwares/OpenMPI/1.10.0/bin/mpif90  CPPFLAGS="-DOMPI_SKIP_MPICXX"  --
      ↪ with-hdf5=$HOME/Softwares/HDF5/1.8.15  --enable-silo  --with-silo=$HOME/Softwares/Silo/4.10.2  --with-samrai=$HOME/
      ↪ Softwares/SAMRAI/2.4.4/linux-dbg  --with-boost=$HOME/Softwares/Boost/1.59.0  --with-eigen=$HOME/Softwares/Eigen/3.2.6  --
      ↪ enable-libmesh  --with-libmesh=$HOME/Softwares/libMesh/0.9.4/linux-dbg  --with-libmesh-method=dbg  --enable-gsl  --with-gsl
      ↪ =$HOME/Softwares/GSL/1.16
make
make check
```

IBAMR from GitHub (`git clone`, the moving `master`), configured against the libMesh install, the same Boost and Eigen, Silo (`--enable-silo`) and GSL (`--enable-gsl`); `make`, `make check`.

OS X Mavericks 10.9.5 install notes (about 2015; unattributed).

OS X MAVERICKS (6/6): THE NOTES' TRAPS

- `wget SAMRAI` is a placeholder (SAMRAI had to be fetched from LLNL by hand).
- The Boost download is saved under the name `download?source=files`, so `tar xvjf boost_1_59_0.tar.bz2` finds no such file; the El Capitan notes of 2016 add the missing `mv` (slide 70).
- Eigen: the tarball is moved into `3.2.6/` before that folder exists, and it unpacks under another name; the Eigen tree's header folder is `Eigen/`, not `eigen` (the same on the Mac's case-insensitive disk, not on Linux).
- GSL and libMesh: `mkdir` without `cd`, so the downloads land in the previous folder.
- PETSc is configured without `--with-c++-support`; PETSc, SAMRAI, libMesh and IBAMR are built in debug form only, and the "debug" IBAMR has no `-g`.
- GSL is configured with both `--disable-static` and `--disable-shared`; libtool then builds the static library anyway.
- The Silo address on `wci.llnl.gov` and the Bitbucket address of Eigen are out of date (Silo is now on GitHub, Eigen on GitLab).

The notes are kept unattributed: the author is not recorded in a form this series can credit.

OS X Mavericks 10.9.5 install notes (about 2015), checked in October 2026.

2016: THE REQUIREMENTS, FROM BOYCE GRIFFITH'S E-MAILS

June 27, 2016 (requirements being updated):

- Boost 1.46.1 or newer, or the bundled 1.60.0, except with libMesh, which needs an external Boost;
- Eigen 3.1.0 or newer, or the bundled 3.2.1 (external with libMesh);
- HDF5 1.8.7 or newer; hypre 2.10b or newer, preferably installed by PETSc;
- libMesh 0.9.5 (optional); PETSc 3.6.x; SAMRAI 2.4.4; Silo 4.9.1 or newer (optional);
- PETSc 3.7 and libMesh 1.0 "soonish".

July 8, 2016:

- IBAMR's GitHub `master` now requires PETSc 3.7.x and libMesh 1.0.0, and Boost 1.57 or newer;
- libMesh 1.0.0 must be configured with `--disable-unique-ptr` (fixed in libMesh's master; expected to be dropped after the next libMesh release).

July 18: a link to IBAMR's wiki page "Building" on GitHub.

> These two e-mails explain the 2016 recipes: Boost 1.57.0 to 1.61.0 built or unpacked by hand, PETSc 3.7.2, libMesh 1.0.0 with `--disable-unique-ptr`.

"July 18 Notes on building IBAMR" (2016): the instructor's notes of Boyce Griffith's e-mails, summarized.

KILLDEVIL FROM GIT MASTERS (2016) (1/6): MODULES, HDF5, SILO, BOOST

"IBAMR Installation 4/16/16" (revised to August 2016) • historical, do not run

```
module add boost/1.55.0   <———not needed since July 2016, since must
      ↪  install boost yourself
module add git/1.8.5.3
module add subversion/1.7.14
module add openmpi_gcc/4.8.1
module add python/2.7.6   <— needed to configure PETSc
tar xvfz hdf5-1.8.16.tar.gz
 cd hdf5-1.8.16
./configure CC=gcc CXX=g++ FC=gfortran F77=gfortran --enable-
      ↪ production --disable-debug --prefix=$HOME/sfw/killdevil/hdf5
      ↪ /1.8.16
make
make check
make install
tar xvfz silo-4.10.2.tar.gz
 cd silo-4.10.2
sh ./configure CC=gcc CXX=g++ FC=gfortran F77=gfortran --prefix=$HOME/
      ↪ sfw/killdevil/silo-4.10.2 --disable-silex
make
make install
tar xvfz boost_1_57_0.tar.gz
(2) cd boost_1_57_0
(3)./bootstrap.sh --prefix=$HOME/sfw/killdevil/boost_1_57_0
(4)./b2 install
```

- The cluster's modules for git, Subversion, the GNU OpenMPI and Python 2.7.6 ("needed to configure PETSc"); the Boost module is dropped from July 2016 ("must install boost yourself"): IBAMR now asked for Boost 1.57 or newer (last slide), and the module had 1.55.0.
- Boost 1.57.0 is built with its own tools, `bootstrap.sh` and `b2`, into `$HOME/sfw/killdevil/boost_1_57_0`.
- The <— arrows are the author's notes, with em dashes, inside the command list.
- Silo is built without HDF5 (no `--with-hdf5`), as in every recipe of this chapter: IBAMR writes its Silo files in Silo's own PDB format.

Nick Battista, "IBAMR Installation 4/16/16" (RTF, revised to August 2016; a PDF print of it is kept beside it).

KILLDEVIL FROM GIT MASTERS (2/6): PETSC'S MASTER FROM BITBUCKET

"IBAMR Installation 4/16/16", PETSc (the home path shown as `/pathto`) • historical, do not run

```
cd sfw/petsc/current
git clone https://bitbucket.org/petsc/petsc.git
(maybe: git clone -b maint https://bitbucket.org/petsc/petsc petsc)
cd petsc
export PETSC_DIR=$PWD
export PETSC_ARCH=killdevil-gcc-opt
./configure \
  --CC=mpicc \
  --CXX=mpicxx \
  --FC=mpif90 \
  --COPTFLAGS="-O3" \
  --CXXOPTFLAGS="-O3" \
  --FOPTFLAGS="-O3" \
  --PETSC_ARCH=$PETSC_ARCH \
  --with-debugging=0 \
  --with-c++-support=1 \
  —-with-hypre=1 \
  --download-hypre=1
make PETSC_DIR=/pathto/sfw/petsc/current/petsc PETSC_ARCH=killdevil-gcc-opt all
make PETSC_DIR=/pathto/sfw/petsc/current/petsc PETSC_ARCH=killdevil-gcc-opt test
make PETSC_DIR=/pathto/sfw/petsc/current/petsc PETSC_ARCH=killdevil-gcc-opt streams
# "( command should be listed in one of the last lines in the terminal :) )"
```

- PETSc from its development repository (then on Bitbucket, now on GitLab), its moving `master`; the `maint` branch is offered as a "maybe".
- In the notes each indented line here, but the hypre one, begins with a non-breaking space (slide 66).
- Only an optimized build, `PETSC_ARCH=killdevil-gcc-opt`, compiled with the module's MPI wrappers.
- `make ... all`, `test`, `streams`: PETSc prints each of these commands at the end of the step before it (configure, then `all`, then `test`), as the note after each says; `streams` measures memory bandwidth.
- —`-with-hypre=1` begins with an em dash: configure does not see `--with-hypre` (slide 66).

Nick Battista, "IBAMR Installation 4/16/16" (2016); a hard-coded home path replaced by `/pathto`.

KILLDEVIL FROM GIT MASTERS (3/6): SAMRAI, OPTIMIZED AND DEBUG

"IBAMR Installation 4/16/16", SAMRAI • historical, do not run

```
(1) put SAMRAI-v2.4.4.tar.gz and SAMRAI-v2.4.4-patch-121212.gz into
      ↪ $HOME/sfw/packages/configures/killdevil/
(2) cd sfw/packages/configures/killdevil
    (3a) tar xvfz SAMRAI-v2.4.4.tar.gz
    (3b) tar xvfz SAMRAI-v2.4.4-patch-121212.gz <<—is this necessary
      ↪ ?!!?
    (3c) cd SAMRAI
    (3d) ./source/scripts/includes —link
    (3e) gunzip -c ../SAMRAI-v2.4.4-patch-121212.gz | patch -p2
# (5a) configure SAMARI opt: ../SAMRAI/configure with the list of
      ↪ slide 17,
#      prefix $HOME/sfw/samrai/2.4.4/killdevil-gcc-opt, no --with-
      ↪ silo
# (5b) make  (5c) make install
# (5d) configure SAMRAI dbg: CFLAGS="-g -O0" CXXFLAGS="-g -O0" FFLAGS
      ↪ ="-g -O0",
#      prefix .../killdevil-gcc-dbg; make; make install
# every indented line of both configures begins with a non-breaking
      ↪ space
```

- The numbered steps are the notes' own. Step (3b) answers its own question: no. A `.gz` patch is not a tar archive, and step (3e) already reads it through `gunzip`.
- Step (3d) has an em dash: `includes —link` in place of `--link`.
- As written, the shell is still inside `SAMRAI` after step (3e), and there `../SAMRAI/configure` is the source tree's own `configure`: the optimized SAMRAI is built in the source tree, and the debug configure (5d) reconfigures the same tree. A build folder beside the source (`mkdir`, `cd`) would avoid both; the notes show none.
- "SAMARI" is the notes' typo; the debug build uses `-g -O0`, no optimization at all.

Nick Battista, "IBAMR Installation 4/16/16" (2016).

KILLDEVIL FROM GIT MASTERS (4/6): LIBMESH AND IBAMR

"IBAMR Installation 4/16/16", libMesh (each indented line begins with a non-breaking space) • historical, do not run

```
cd sfw/killdevil/libmesh/master-opt
git clone https://github.com/libMesh/libmesh.git
cd libmesh
./configure --disable-maintainer-mode \
  --prefix=$HOME/sfw/killdevil/libmesh/master-opt \
  --with-methods=opt \
  PETSC_DIR=$HOME/sfw/petsc/current/petsc \
  PETSC_ARCH=killdevil-gcc-opt \
  CC=mpicc \
  CXX=mpicxx \
  CFLAGS="-O3" \
  CXXFLAGS="-O3" \
  --with-boost=$HOME/sfw/killdevil/boost_1_57_0 \
  --disable-cxx11 \
  --disable-openmp \
  --disable-perflog \
  --disable-strict-lgpl \
  --disable-pthreads \
  --disable-cppthreads \
  --disable-unique-ptr
make
make install
```

"IBAMR Installation 4/16/16", IBAMR (the same) • historical, do not run

```
(2) git IBAMR: git clone https://github.com/IBAMR/IBAMR.git
(3) cd IBAMR
export PETSC_ARCH=killdevil-gcc-opt
export PETSC_DIR=$HOME/sfw/petsc/current/petsc
$HOME/sfw/IBAMR/configure --config-cache \
  CC=mpicc \
  CXX=mpicxx \
  FC=gfortran \
  CFLAGS="-O3" \
  CXXFLAGS="-O3" \
  FFLAGS="-O3" \
  --with-hypre=$PETSC_DIR/$PETSC_ARCH \
  --with-boost=$HOME/sfw/killdevil/boost_1_57_0 \
  --with-hdf5=$HOME/sfw/killdevil/hdf5/1.8.16 \
  --with-samrai=$HOME/sfw/samrai/2.4.4/killdevil-gcc-opt \
  --with-silo=$HOME/sfw/killdevil/silo-4.10.2 \
  --enable-libmesh \
  --with-libmesh=$HOME/sfw/killdevil/libmesh/master-opt \
  --with-libmesh-method=opt
```

libMesh's and IBAMR's `master` branches; libMesh with `--disable-unique-ptr`, which the July 2016 e-mail asked for libMesh 1.0.0 (it says libMesh's `master`, cloned here, already had the fix). IBAMR is configured inside its source tree, and the notes stop at configure: no `make`, `make lib` or `make check`.

Nick Battista, "IBAMR Installation 4/16/16" (2016).

KILLDEVIL FROM GIT MASTERS (5/6): “COPY IT INTO AN EMAIL IN GMAIL”: WHAT WAS REALLY WRONG

The notes record the same problem four times, twice in these words: “for some reason copying and pasting this straight from this document does not seem to work, I copied it into an email in gmail, and then copy and pasted it and it worked fine.”

The cause, almost certainly: non-breaking spaces. Every indented line of the SAMRAI, libMesh and IBAMR configures, and all but one of PETSc’s, begins with a non-breaking space, 86 in all. The shell hands each one to `configure` as an extra argument, which an autotools `configure` takes for a machine type, and it stops (`invalid host type`). Copied through a web page such as Gmail, non-breaking spaces become ordinary spaces, so the command then worked. The em dashes (`—-with-hypre=1`, `includes —link`, the prose `—enable-implicit-template-instantiation”`) are a second trap, which an e-mail does not repair; none of them is in a block that carries the Gmail note. A plain-text editor shows both.

Nick Battista, “IBAMR Installation 4/16/16” (RTF and its PDF print, 2016), checked in October 2026.

KILLDEVIL FROM GIT MASTERS (6/6): THE NOTES' OTHER SLIPS

- `tar xvfz` on the patch (slide 64);
- SAMRAI configured without `--with-silo` (harmless: IBAMR does not use SAMRAI's Silo restart files);
- IBAMR configured in its source tree, and no `make` for it;
- PETSc and libMesh from moving `master` branches: the build cannot be repeated;
- the Bitbucket address of PETSc is obsolete (PETSc is on GitLab);
- a hard-coded home path in the PETSc `make` lines;
- typos "SAMARI" and "it's master branch".

The PDF print of the notes also wraps long lines in the middle of options (e.g. `--enable-` / `production`).

Nick Battista, "IBAMR Installation 4/16/16" (RTF and its PDF print, 2016), checked in October 2026.

2016–17: SWITCHING BETWEEN THE MODULES AND YOUR OWN INSTALL (1/2)

"To use the old one, you need to module initadd ibamr/3803. Make sure that you module initrm other ibamr and open_mpi modules. To use the new one, you need to module initadd ibamr/2016-11. Make sure that you module initrm other ibamr and openmpi. To use your own install, you need to remove all ibamr modules and add these modules –" "How to update examples to new install", slide 3 • historical, do not run

```
module add openmpi_gcc/4.8.1
module add python
module add git
```

slide 4, the Makefile for the new module • historical, do not run

```
IBAMR_SRC_DIR = /nas02/apps/ibamr-2016-11/OFED-1.5.2/sfw/ibamr/
      ↪ IBAMR
IBAMR_BUILD_DIR = /nas02/apps/ibamr-2016-11/OFED-1.5.2/sfw/ibamr/
      ↪ ibamr-objs-opt
```

- In November 2016 UNC installed a second module, `ibamr/2016-11`; "Module 2016-11 is the newest killdevil module. 3803 is old."
- `module initadd` and `initrm` change the modules loaded at every login; two IBAMR or two OpenMPI modules at once give mixed libraries.
- Nick Battista moved the IB examples to the new module early in 2017 (the code changes: chapter X3); his IBFE ports still failed to compile on February 3, and a week later the course's to-do list reports "All IBFE examples are updated and work the new module."

The instructor's "How to update examples to new install" (2017), slides 3–4; the Spring 2017 to-do list; the IBFE read-me; the 2016-11 example Makefiles (a host name in a comment replaced).

2016–17: SWITCHING BETWEEN THE MODULES AND YOUR OWN INSTALL (2/2)

The 2016-11 example Makefiles list four install locations, three of them commented out: the 3803 module (“Feb. 2014 (?)”), the 2016-11 module (“Nov 2016”), a home install on KillDevil (“KD (NOT MODULE)”, `$(HOME)/sfw2/...`) and the lab’s THLinux machines (`/usr/bin/sfw/ibamr/...`): a short history of this chapter.

> Every era of this chapter left a line in those comments; chapter X3 (slides on dating old code) reads them the other way round, to tell which IBAMR an old program was written for.

The 2016-11 example Makefiles (a host name in a comment replaced).

EL CAPITAN, 2016 (1/7): MACPORTS GCC, BOOST 1.61.0, HDF5 1.8.17

"IBAMR install on OS X El Capitan Version 10.11.6, August 17, 2016" • historical, do not run

```
sudo port install gcc49
cd $HOME
mkdir sfw
cd sfw
mkdir linux
cd $HOME/sfw/linux
mkdir boost
cd boost/
wget http://sourceforge.net/projects/boost/files/latest/
      ↪ download?source=files
mv download\?source\=files boost_1_61_0.tar.bz2
tar xvjf boost_1_61_0.tar.bz2
mv boost_1_61_0 1.61.0
cd 1.61.0/
export BOOST_ROOT=$HOME/sfw/linux/boost/1.61.0/
```

the same notes, HDF5 (␣: spaces after the backslash) • historical, do not run

```
cd $HOME/sfw/linux
wget http://www.hdfgroup.org/ftp/HDF5/current/src/hdf5-1.8.17.
      ↪ tar
tar xvf hdf5-1.8.17.tar
cd hdf5-1.8.17
./configure   \
CC=gcc-mp-4.9   \
CXX=g++-mp-4.9 \␣␣
FC=gfortran-mp-4.9 \␣␣
F77=gfortran-mp-4.9  \␣
--enable-production   \
--disable-debug   \
--prefix=$HOME/sfw/linux/hdf5/1.8.17
make
make check
make install
```

Boost is only unpacked (header-only), renamed by hand from "latest" to 1.61.0, and announced to later configures with `BOOST_ROOT`. HDF5: the trap of slide 16 three times. In `CXX=g++-mp-4.9 \␣␣` the backslash quotes one space, so `configure` receives a stray argument made of a single space; it takes it for a machine type and stops with an error (`invalid host type`). The `FC=` and `F77=` lines then fail as commands of their own, the three option lines run together as one more failing command, and `make` finds no `Makefile`.

"IBAMR install on OS X El Capitan Version 10.11.6, August 17, 2016" (RTF; unattributed). The trailing spaces were checked in the RTF source.

EL CAPITAN (2/7): SILO 4.10.2 AND OPENMPI 1.10.2

El Capitan notes, Silo • historical, do not run

```
cd $HOME/sfw/linux
wget https://wci.llnl.gov/content/assets/docs/simulation/
      ↪ computer-codes/silo/silo-4.10.2/silo-4.10.2.tar.gz
tar xvfz silo-4.10.2.tar.gz
cd silo-4.10.2
./configure   \
CC=gcc-mp-4.9 \
CXX=g++-mp-4.9 \
FC=gfortran-mp-4.9 \
F77=gfortran-mp-4.9 \␣␣␣
--prefix=$HOME/sfw/linux/silo/4.10.2   \
--disable-silex
make
make check
make install
```

“Mac OS X comes with zlib”, the notes say, so Silo needs no separate zlib here (contrast 2019, slide 90). The `F77=` line has the same trap: Silo’s `configure` stops too.

El Capitan notes, OpenMPI • historical, do not run

```
cd $HOME/sfw/linux/
wget http://www.open-mpi.org/software/ompi/v1.10/downloads/
      ↪ openmpi-1.10.2.tar.gz
tar xvfz openmpi-1.10.2.tar.gz
cd openmpi-1.10.2
./configure \
CC=gcc-mp-4.9 \
CXX=g++-mp-4.9 \
FC=gfortran-mp-4.9 \
F77=gfortran-mp-4.9 \
--prefix=$HOME/sfw/linux/openmpi/1.10.2  \
--disable-mpi-cxx-seek \
--disable-heterogeneous \
--enable-orterun-prefix-by-default
make
make check
make install
```

“IBAMR install on OS X El Capitan Version 10.11.6, August 17, 2016” (unattributed).

El Capitan (3/7): PETSc 3.7.2, debug and optimized; SAMRAI

El Capitan notes, PETSc debug • historical, do not run

```
cd $HOME/sfw
mkdir petsc
cd petsc
wget http://ftp.mcs.anl.gov/pub/petsc/release-snapshots/petsc-3.7.2.tar.
      ↪ gz
tar xvfz petsc-3.7.2.tar.gz
mv petsc-3.7.2 3.7.2
cd 3.7.2/
export PETSC_DIR=$PWD
export PETSC_ARCH=linux-debug
./config/configure.py   \
--CC=$HOME/sfw/linux/openmpi/1.10.2/bin/mpicc   \
--CXX=$HOME/sfw/linux/openmpi/1.10.2/bin/mpicxx   \
--FC=$HOME/sfw/linux/openmpi/1.10.2/bin/mpif90   \
--LDFLAGS="-L$HOME/sfw/linux/openmpi/1.10.2/lib -Wl,-rpath,$HOME/sfw/
      ↪ linux/openmpi/1.10.2/lib"   \
--with-default-arch=0   \
--PETSC_ARCH=$PETSC_ARCH   \
--with-debugging=1   \
--with-c++-support   \
--with-hypre=1   \
--download-hypre=1
make
make test
```

- PETSc 3.7.2, the version IBAMR's master required from July 2016 (slide 61). The spaces before these backslashes are harmless: only a space *after* a backslash breaks a line.
- Optimized: `export PETSC_ARCH=linux-opt`; adds `--COPTFLAGS="-O3" --CXXOPTFLAGS="-O3" --FOPTFLAGS="-O3"`, `--with-debugging=0`, and ends `--download-hypre=1 --with-x=0`; `make`, `make test`.
- SAMRAI: "*download SAMRAI-v2.4.4.tar.gz from http://computation.llnl.gov/projects/samrai/software"; the patch from IBAMR's GitHub release, debug and optimized (slide 21).

"IBAMR install on OS X El Capitan Version 10.11.6, August 17, 2016" (unattributed).

EL CAPITAN (4/7): LIBMESH 1.0.0, CONFIGURED FOR DEBUGGING

El Capitan notes, libMesh debug • historical, do not run

```
cd $HOME/sfw/linux
mkdir libmesh
cd libmesh
mkdir 1.0.0
cd 1.0.0/
wget https://github.com/libMesh/libmesh/releases/download/v1
      ↪ .0.0/libmesh-1.0.0.tar.gz
tar xvfz libmesh-1.0.0.tar.gz
mv libmesh-1.0.0 LIBMESH
cd $HOME/sfw/linux/libmesh/1.0.0
mkdir objs-debug
cd objs-debug/
export BOOST_ROOT=$HOME/sfw/linux/boost/1.61.0/
../LIBMESH/configure \
  --prefix=$HOME/sfw/linux/libmesh/1.0.0/1.0.0-debug \
  --with-methods=dbg \
  PETSC_DIR=$HOME/sfw/petsc/3.7.2 \
  PETSC_ARCH=linux-debug \
```

continued • historical, do not run

```
  CC=$HOME/sfw/linux/openmpi/1.10.2/bin/mpicc \
  CXX=$HOME/sfw/linux/openmpi/1.10.2/bin/mpicxx \
  FC=$HOME/sfw/linux/openmpi/1.10.2/bin/mpif90 \
  F77=$HOME/sfw/linux/openmpi/1.10.2/bin/mpif90 \
  --enable-triangle \
  --disable-cxx11 \
  --disable-openmp \
  --disable-perflog \
  --disable-strict-lgpl \
  --disable-pthreads \
  --disable-cppthreads \
  --disable-unique-ptr
make
make install
```

libMesh is configured in a build folder (`objs-debug`) beside its source (`LIBMESH`) and installed into `1.0.0-debug`, with `--disable-unique-ptr` as the July 2016 e-mail asked and Triangle for 2D meshes.

"IBAMR install on OS X El Capitan Version 10.11.6, August 17, 2016" (unattributed).

EL CAPITAN (5/7): HIDING LIBMESH'S BUNDLED BOOST; THE OPTIMIZED LIBMESH

"**Note: If using libMesh, it is necessary to use an external boost with IBAMR. Change the name of the boost directory in the include folder so IBAMR uses the external library:" El Capitan notes, after `make install` •

historical, do not run

```
cd $HOME/sfw/linux/libmesh/1.0.0/1.0.0-debug/include
mv boost/ boost_name_change
```

- libMesh 1.0.0 installs its own copy of Boost in its `include` folder; the notes rename that folder so that IBAMR uses the external Boost, as the June 2016 e-mail required.
- Optimized libMesh: `cd sfw/linux/libmesh/1.0.0` (a relative path), `objs-opt`, the prefix `.../1.0.0-opt`, `--with-methods=opt`, `PETSC_ARCH=linux-opt`, the same options; then the same `mv` in `1.0.0-opt/include`.

"IBAMR install on OS X El Capitan Version 10.11.6, August 17, 2016" (unattributed).

EL CAPITAN (6/7): IBAMR, DEBUG AND OPTIMIZED, AND EX0

El Capitan notes, IBAMR debug • historical, do not run

```
cd $HOME/sfw
mkdir ibamr
cd ibamr
git clone https://github.com/IBAMR/IBAMR.git
mkdir ibamr-objs-dbg
cd ibamr-objs-dbg
export BOOST_ROOT=$HOME/sfw/linux/boost/1.61.0/
export PETSC_ARCH=linux-debug
export PETSC_DIR=$HOME/sfw/petsc/3.7.2
../IBAMR/configure \
  CFLAGS="-g -O1 -Wall" \
  CXXFLAGS="-g -O1 -Wall" \
  FCFLAGS="-g -O1 -Wall" \
  CC=$HOME/sfw/linux/openmpi/1.10.2/bin/mpicc \
  CXX=$HOME/sfw/linux/openmpi/1.10.2/bin/mpicxx \
  FC=$HOME/sfw/linux/openmpi/1.10.2/bin/mpif90 \
  CPPFLAGS="-DOMPI_SKIP_MPICXX" \
```

continued • historical, do not run

```
  --with-hypre=$PETSC_DIR/$PETSC_ARCH \
  --with-samrai=$HOME/sfw/samrai/2.4.4/linux-g++-debug \
  --with-hdf5=$HOME/sfw/linux/hdf5/1.8.17 \
  --with-silo=$HOME/sfw/linux/silo/4.10.2 \
  --with-boost=$HOME/sfw/linux/boost/1.61.0 \
  --enable-libmesh \
  --with-libmesh=$HOME/sfw/linux/libmesh/1.0.0/1.0.0-debug \
  --with-libmesh-method=dbg
make lib
make examples
cd $HOME/sfw/ibamr/ibamr-objs-debug/examples/IB/explicit/ex0
./main2d input2d
```

Optimized: `ibamr-objs-opt`; `CC`, `CXX`, `F77`, `FC`, `MPICC`, `MPICXX` the OpenMPI 1.10.2 wrappers; `CFLAGS="-O3 -Wall" CXXFLAGS="-O3 -Wall -std=gnu++11" FFLAGS="-O3 -Wall" FCFLAGS="-O3 -Wall" CPPFLAGS="-DOMPI_SKIP_MPICXX"`; `linux-g++-opt`, `1.0.0-opt`, `--with-libmesh-method=opt`; `make`, `make check`, `make examples`; then `cd $HOME/sfw/ibamr/ibamr-objs-opt/examples/IB/explicit/ex0` and `./main2d input2d`. The example path is the new one: `IB/explicit/ex0`.

"IBAMR install on OS X El Capitan Version 10.11.6, August 17, 2016" (unattributed).

EL CAPITAN (7/7): THE NOTES' SLIPS

- The debug build folder is created as `ibamr-objs-dbg`, but the run goes to `ibamr-objs-debug`.
- Spaces after the continuation backslash in three HDF5 lines and one Silo line (slides 70–71): those configures stop early.
- `cd sfw/linux/libmesh/1.0.0` (relative) before the optimized libMesh works only from the home folder.
- Boost's "latest" download is renamed to 1.61.0 by hand: correct in August 2016, wrong from September 28, 2016, when Boost 1.62.0 came out.
- libMesh's own Boost is hidden by renaming a folder of the install (`boost_name_change`), a change made by hand inside an installed library.

These notes are kept unattributed: they were shared from another lab account.

"IBAMR install on OS X El Capitan Version 10.11.6, August 17, 2016" (unattributed), checked in October 2026.

“LAURA’S INSTALL” (KILLDEVIL, 2016–17) (1/9): MODULES, BOOST 1.60.0, EIGEN

“Laura’s Install” (KillDevil Modules) • historical, do not run

```
*remove ibamr/3803 module*
module add openmpi_gcc/4.8.1
module add python
module add git
cd $HOME
mkdir sfw
cd sfw
mkdir linux
```

Boost; Eigen • historical, do not run

```
*Boost*
cd $HOME/sfw/linux
mkdir boost
cd boost
*Put boost file in the boost folder*
tar xvfz boost_1_60_0.tar.gz
mv boost_1_60_0 1.6.0
export BOOST_ROOT=$HOME/sfw/linux/boost/1.6.0
cd 1.6.0
./bootstrap.sh --prefix=$HOME/sfw/linux/boost/1.6.0
./b2 install
ln -s $PWD/boost $PWD/include/boost
Eigen
cd $HOME/sfw/linux
mkdir eigen
cd eigen
*Put eigen file in the eigen folder*
tar xvjf eigen-eigen-b30b87236a1b.tar.bz2
```

“Laura’s Install” (the instructor, KillDevil, Sep 2016 – Apr 2017; kept as a 2023 export of the Google document).

“LAURA’S INSTALL” (2/9): WHAT THE FIRST STEPS SAY

- The instructor’s own KillDevil install of 2016–17, written as a Google document; a copy titled “IBAMR Install Notes” (November 2016) adds “Please use the exact versions of all libraries.”
- The stars mark instructions, not commands: `*remove ibamr/3803 module*`, `*Put boost file in the boost folder*`.
- Boost 1.60.0 is built with `bootstrap.sh` and `b2` into a folder named `1.6.0` (a slip for 1.60.0); the `ln -s` line is Boyce Griffith’s workaround (slide 86), redundant here because `b2 install` has already made `include/boost`.
- Eigen is the 3.2.7 release, as Bitbucket’s archive named by its Mercurial changeset (`b30b87236a1b`), unpacked and used in place.
- The downloads it expects to be “put” in place, and PETSc’s, were kept beside it (slide 85).

“Laura’s Install” (the instructor, KillDevil, 2016–17); “IBAMR Install Notes” (November 2016).

“LAURA’S INSTALL” (3/9): HDF5 1.8.16 AND SILO 4.10

“Laura’s Install”, HDF5 • historical, do not run

```
Hdf5
cd $HOME/sfw/linux
*put file in linux folder*
tar xvjf hdf5-1.8.16.tar.bz2
cd hdf5-1.8.16
./configure \
  CC=gcc \
  CXX=g++ \
  FC=gfortran \
  F77=gfortran \
  --enable-production \
  --disable-debug \
  --prefix=$HOME/sfw/linux/hdf5/1.8.16
make
make check
make install
```

“Laura’s Install”, Silo • historical, do not run

```
Silo
cd $HOME/sfw/linux
*Put silo .tar.gz in the linux folder*
tar xvfz path/to/silo-4.10.tar.gz
cd silo-4.10
./configure \
  CC=gcc \
  CXX=g++ \
  FC=gfortran \
  F77=gfortran \
  --prefix=$HOME/sfw/linux/silo/4.10 \
  --disable-silex
make
make install
```

The patterns of 2012 with newer versions and the GNU compilers of the `openmpi_gcc/4.8.1` module. `path/to/` is a placeholder: the file was “put” in the current folder one line before.

“Laura’s Install” (the instructor, KillDevil, 2016–17).

“LAURA’S INSTALL” (4/9): PETSC 3.7.2, OPTIMIZED, WITH HYPRE

“Laura’s Install”, PETSc • historical, do not run

```
Petsc
module load python
cd $HOME/sfw
mkdir petsc
cd petsc
wget http://ftp.mcs.anl.gov/pub/petsc/release-snapshots/
      ↪ petsc-3.7.2.tar.gz
tar xvfz petsc-3.7.2.tar.gz
mv petsc-3.7.2 3.7.2
cd 3.7.2
export PETSC_DIR=$PWD
export PETSC_ARCH=killdevil-gcc-opt
```

- The 2016 KillDevil notes’ PETSc step (slide 63), with the 3.7.2 release instead of the moving `master`, and the `make` lines with `$HOME` instead of a fixed path.
- The same em dash in `—-with-hypre=1`, carried over from those notes; `--download-hypre=1` on the next line still asks for hypre.
- Only the optimized build (`killdevil-gcc-opt`): this install is for runs.

continued • historical, do not run

```
./configure \
  --CC=mpicc \
  --CXX=mpicxx \
  --FC=mpif90 \
  --COPTFLAGS="-O3" \
  --CXXOPTFLAGS="-O3" \
  --FOPTFLAGS="-O3" \
  --PETSC_ARCH=$PETSC_ARCH \
  --with-debugging=0 \
  --with-c++-support=1 \
  —-with-hypre=1 \
  --download-hypre=1
make PETSC_DIR=$HOME/sfw/petsc/3.7.2 PETSC_ARCH=killdevil-gcc-opt all
make PETSC_DIR=$HOME/sfw/petsc/3.7.2 PETSC_ARCH=killdevil-gcc-opt
      ↪ test
make PETSC_DIR=$HOME/sfw/petsc/3.7.2 PETSC_ARCH=killdevil-gcc-opt
      ↪ streams
```

“Laura’s Install” (the instructor, KillDevil, 2016–17).

“LAURA’S INSTALL” (5/9): SAMRAI, DEBUG AND OPTIMIZED

“Laura’s Install”, SAMRAI • historical, do not run

```
SAMRAI
cd $HOME/sfw
mkdir samrai
cd samrai
mkdir 2.4.4
cd 2.4.4
*put SAMRAI file in 2.4.4. folder*
tar xvfz SAMRAI-v2.4.4.tar.gz
mv SAMRAI SAMRAI-2.4.4
cd SAMRAI-2.4.4
./source/scripts/includes --link
*Put patch in SAMRAI-2.4.4 folder*
gunzip -c SAMRAI-v2.4.4-patch-121212.gz | patch -p2
cd $HOME/sfw/samrai/2.4.4
mkdir objs-debug
cd objs-debug
# the debug configure of slide 17, exactly;
# then objs-opt with the optimized one
```

- This recipe is the source of the configure line on slide 18.
- The patch is put *inside* `SAMRAI-2.4.4`, so the `gunzip` needs no `../`: a fix for the wrong-folder slip of 2012–13, done by moving the file instead of the command.
- SAMRAI is built twice, although PETSc, libMesh and IBAMR are built only optimized; the debug SAMRAI is not used by this recipe.

“Laura’s Install” (the instructor, KillDevil, 2016–17).

“LAURA’S INSTALL” (6/9): LIBMESH 1.0.0

“Laura’s Install”, libMesh • historical, do not run

```
Libmesh
cd $HOME/sfw/linux
mkdir libmesh
cd libmesh
mkdir 1.0.0
cd 1.0.0
*Put libmesh-1.0.0 file in this folder*
tar xvfz libmesh-1.0.0.tar.gz
mv libmesh-1.0.0 LIBMESH
mkdir objs-opt
cd objs-opt
export BOOST_ROOT=$HOME/sfw/linux/boost/1.6.0
../LIBMESH/configure --disable-maintainer-mode \
  --prefix=$HOME/sfw/linux/libmesh/1.0.0/LIBMESH/master-opt \
  --with-methods=opt \
  --with-boost=$HOME/sfw/linux/boost/1.6.0 \
  --with-eigen=$HOME/sfw/linux/eigen/eigen-eigen-b30b87236a1b \
  PETSC_DIR=$HOME/sfw/petsc/3.7.2 \
  PETSC_ARCH=killdevil-gcc-opt \
  CC=mpicc \
```

continued • historical, do not run

```
  CXX=mpicxx \
  FC=gfortran \
  CFLAGS="-O3" \
  CXXFLAGS="-O3" \
  FFLAGS="-O3" \
 --enable-triangle \
  --disable-cxx11 \
  --disable-openmp \
  --disable-perflog \
  --disable-strict-lgpl \
  --disable-pthreads \
  --disable-cppthreads \
  --disable-unique-ptr
make
make install
```

libMesh 1.0.0 with `--disable-unique-ptr` (the July 2016 e-mail) and Triangle for 2D meshes. It is installed inside its own source folder (`.../LIBMESH/master-opt`) and named “master” although it is the 1.0.0 release; `--disable-cxx11` here while IBAMR is compiled with `-std=gnu++11` (next slide) worked, but mixes two language standards.

“Laura’s Install” (the instructor, KillDevil, 2016–17).

“LAURA’S INSTALL” (7/9): IBAMR’S MASTER

“Laura’s Install”, IBAMR • historical, do not run

```
IBAMR
module load git
cd $HOME/sfw
mkdir ibamr
cd ibamr
git clone https://github.com/IBAMR/IBAMR.git
cd $HOME/sfw/ibamr
mkdir ibamr-objs-opt
cd ibamr-objs-opt
export PETSC_DIR=$HOME/sfw/petsc/3.7.2
export  PETSC_ARCH=killdevil-gcc-opt
export BOOST_ROOT=$HOME/sfw/linux/boost/1.6.0
../IBAMR/configure  --config-cache \
  CC=mpicc \
  CXX=mpicxx \
  FC=gfortran \
  F77=gfortran \
  MPICC=mpicc \
  MPICXX=mpicxx \
  CFLAGS="-O3 -Wall" \
  CXXFLAGS="-O3 -Wall -std=gnu++11" \
  FFLAGS="-O3 -Wall" \
  FCFLAGS="-O3 -Wall" \
  CPPFLAGS="-DOMPI_SKIP_MPICXX" \
```

continued • historical, do not run

```
  --with-hypre=$PETSC_DIR/$PETSC_ARCH \
  --with-samrai=$HOME/sfw/samrai/2.4.4/linux-g++-opt \
  --with-hdf5=$HOME/sfw/linux/hdf5/1.8.16 \
  --with-silo=$HOME/sfw/linux/silo/4.10 \
  --with-boost=$HOME/sfw/linux/boost/1.6.0 \
  --with-eigen=$HOME/sfw/linux/eigen/eigen-eigen-b30b87236a1b \
  --enable-libmesh \
  --with-libmesh=$HOME/sfw/linux/libmesh/1.0.0/LIBMESH/master-
      ↪ opt \
  --with-libmesh-method=opt
make lib
make check
```

IBAMR’s `master` of late 2016, unpinned; `-std=gnu++11` asks the compiler for C++11 with GNU extensions.

“Laura’s Install” (the instructor, KillDevil, 2016–17).

“LAURA’S INSTALL” (8/9): THE FIRST RUN, FROM A PATH THE 2016 IBAMR NO LONGER HAD

“Laura’s Install”, after `make check` • historical, do not run

```
cd $HOME/sfw/ibamr/ibamr-objs-opt/examples/IB/staggered_grid/explicit/ex0
make examples
./main2d input2d
```

The run uses the 2012 example path (`IB/staggered_grid/explicit/ex0`), which IBAMR no longer had: its examples were in `IB/explicit` by its first GitHub release (2014), as the El Capitan notes of the same year use. So the `cd` fails, `make examples` builds every example from the top of the build folder, and `./main2d` is not found. The KillDevil module `ibamr/2016-11` dates from the same November.

“Laura’s Install” (the instructor, KillDevil, 2016–17).

“LAURA’S INSTALL” (9/9): THE FILES BESIDE IT, AND ITS OWN SLIPS

Kept beside the recipe (“Install Files”): every download it uses, eight files of about 168 MB added in September and November 2016: Boost 1.60.0, the Eigen archive `b30b87236a1b` (1,155,162 bytes), HDF5 1.8.16, `silo-4.10.tar.gz` (13,134,261 bytes), PETSc 3.7.2, SAMRAI 2.4.4, `SAMRAI-v2.4.4-patch-121212.gz` (82,467 bytes) and libMesh 1.0.0. A copy titled “Copy of Laura’s Install (In Progress)” (November 2016) has the same commands; only its headings differ (“*Modules*” in place of “*Killdevil Modules*”, no “*Install Instructions*”, two star separator lines in place of five).

The recipe’s slips

- —`-with-hypre=1`: an em dash (slide 80);
- soft line breaks: the lines of most command blocks are soft line breaks inside one paragraph per block (163 soft breaks in 33 paragraphs), so readers that ignore soft breaks run the commands together;
- Boost 1.60.0 in a folder named `1.6.0`;
- the redundant `ln -s` after `b2 install`;
- libMesh installed inside its source folder and named “master”;
- `--disable-cxx11` for libMesh, `-std=gnu++11` for IBAMR;
- the 2012 example path, absent from the 2016 IBAMR; `tar xvfz path/to/silo-4.10.tar.gz`;
- the appendix leftovers; IBAMR’s `master` not pinned.

“Laura’s Install” and its copy (2016–17); the Install Files folder (listed on the course’s Drive, not unpacked), checked in October 2026.

BOYCE GRIFFITH'S BOOST INCLUDE-FOLDER WORKAROUND (2016)

From the appendix of "Laura's Install", "Random Misc. Notes for Laura" (Boyce Griffith's advice, summarized):

- unpack Boost (his example: `boost_1_62_0`, in a `boost` folder of his software tree);
- libMesh expects `<boost folder>/include` to contain the headers, which a Boost download does not have; so, inside `boost_1_62_0`, make `include` and link the headers into it;
- give the same `--with-boost=` to libMesh and to IBAMR.

the workaround, as summarized (his home path shown as `/pathto`) •

historical, do not run

```
cd /pathto/sfw/boost/boost_1_62_0
mkdir include
ln -s $PWD/boost $PWD/include/boost
./configure --with-boost=$HOME/sfw/boost/boost_1_62_0   #
      ↪ libMesh
```

The same appendix keeps a stray SAMRAI flag list and a second, shorter IBAMR configure: `CC=mpicc CXX=mpicxx FC=gfortran CFLAGS="-O3" CXXFLAGS="-O3" FFLAGS="-O3"`, no `MPICC` or `MPICXX`, no `CPPFLAGS`, no `-std=gnu++11`, the same `--with-...` options; then `make` and `make check`.

The 2019 recipe uses the same trick (slide 89).

"Laura's Install", appendix "Random Misc. Notes for Laura" (2016–17; Boyce Griffith's e-mailed advice, summarized; his home path replaced).

DOGWOOD (2018–20): THE MODULES IBAMR/2018-03 AND IBAMR/0.6.0

"Run your IBAMR simulation on Dogwood" (2018), slides 11 and 23 (prompt scrubbed) • historical, do not run

```
# "Ibamr is now installed on dogwood, the module is ibamr
      ↪ /2018-03!"
[<user>@<host> 769_examples]$ module add ibamr/2018-03
[<user>@<host> 769_examples]$ module list
Currently Loaded Modules:
  1) openmpi_2.0.3/gcc_4.8.5   2) ibamr/2018-03
# In the Makefile:
IBAMR_SRC_DIR = /nas/longleaf/apps-dogwood/ibamr/2018-03/sfw/ibamr
      ↪ /IBAMR
IBAMR_BUILD_DIR  =/nas/longleaf/apps-dogwood/ibamr/2018-03/sfw/
      ↪ ibamr/ibamr-objs-opt
```

IBFE lecture 5, "Plans for June" (2020), slides 2–4 • historical, do not run

```
module add ibamr          # the old one: ibamr/2018-03
module add ibamr/0.6.0
module rm ibamr
```

- Dogwood was UNC's cluster for large parallel jobs after KillDevil, with Slurm in place of LSF (chapter X1). In 2018 its IBAMR module, `ibamr/2018-03`, was a snapshot of IBAMR from before the 0.3.0 release (September 2018), loaded together with the module `openmpi_2.0.3/gcc_4.8.5`.
- In June 2020 a second module, `ibamr/0.6.0`, held IBAMR 0.6.0 (released December 2019); plain `module add ibamr` still gave the old one. The lecture added: "I recommend installing your own updated version of ibamr as well", and planned to keep the old examples in a folder `ibamr-examples-2018-03`.
- The porting work that the 2018 module required is chapter X3's subject.

The instructor's "Run your IBAMR simulation on Dogwood" slides 11 and 23 (2018–20); IBFE lecture 5, "Plans for June" (June 2020); IBAMR's release tags.

DOGWOOD, 2019 (1/9): THE RECIPE AT A GLANCE

The last hand-built install in the course's files: a PDF written in July 2019 and added to the course folder in November 2019, "IBAMR Install", which lists its commands in the order they were run. Its list of versions ("I used the following library/package versions:") names `boost_1_70_0`, `eigen-3.3.7`, `hdf5-1.10.5`, `SAMRAI-v2.4.4`, `silo-4.10.2` and `zlib-1.2.11`.

The PDF is not signed; it is kept in the repository's `originals` folder.

step	version
shell, MPI	`bash`; module `openmpi_2.0.3/gcc_4.8.5`
Boost	1.70.0, headers only
Eigen	3.3.7 (unpacked, not used)
HDF5	1.10.5
zlib, Silo	1.2.11, 4.10.2
PETSc	3.10.5, optimized and debug, with hypre
SAMRAI	2.4.4 with `ibamr-samrai-fixes.patch`
libMesh	1.4.1, debug and optimized
IBAMR	v0.4.1 (a GitHub release), debug and optimized

"IBAMR Install" / "IBAMR Install November 2019" (PDF of July 2019, uploaded November 2019; author not stated).

DOGWOOD, 2019 (2/9): BOOST 1.70.0 AND EIGEN 3.3.7

"IBAMR Install" (2019), page 1 • historical, do not run

```
bash
module add openmpi_2.0.3/gcc_4.8.5
cd $HOME/sfw/linux
mkdir boost
cd boost
tar xvfz path/to/boost_1_70_0.tar.gz
mv boost_1_70_0 1.7.0
export BOOST_ROOT=$HOME/sfw/linux/boost/1.7.0
mkdir $BOOST_ROOT/include
ln -s $BOOST_ROOT/boost $BOOST_ROOT/include
cd $HOME/sfw
tar -xvf eigen-eigen-323c052e1731.tar.bz2
mv eigen-eigen-323c052e1731 eigen-3.3.7
mkdir eigen
mv eigen-3.3.7/ eigen/
```

- `bash` first: the recipe switches to the bash shell before anything else.
- Boost 1.70.0 is only unpacked, into a folder named `1.7.0`, and given an `include` folder with a link to its headers: Boyce Griffith's 2016 workaround (slide 86).
- Eigen 3.3.7 (a snapshot named by its Mercurial hash) is unpacked to `$HOME/sfw/eigen/eigen-3.3.7` but never passed to libMesh or IBAMR, so IBAMR uses the copy it bundles.
- `path/to/` is a placeholder.

"IBAMR Install" (2019), page 1.

DOGWOOD, 2019 (3/9): HDF5 1.10.5, ZLIB 1.2.11 AND SILO 4.10.2

"IBAMR Install" (2019), pages 1–2 • historical, do not run

```
mkdir hdf5
cd hdf5
tar xvjf $HOME/hdf5-1.10.5.tar.bz2
cd hdf5-1.10.5/
./configure CC=gcc CXX=g++ FC=gfortran F77=gfortran --enable-
      ↪ build-mode=production --prefix=$HOME/sfw/linux/hdf5
      ↪ /1.10.5
make -j4
make -j4 check
make -j4 install
cd
cd sfw/
cd linux/
mkdir zlib
cd zlib/
tar xvf $HOME/zlib-1.2.11.tar
mv zlib-1.2.11/ 1.2.11
cd 1.2.11
./configure --prefix=$HOME/sfw/linux/zlib/1.2.11
make install
```

continued • historical, do not run

```
cd
cd sfw/linux/
tar xvf $HOME/silo-4.10.2.tar
cd silo-4.10.2/
 ./configure CC=gcc CXX=g++ FC=gfortran F77=gfortran --prefix=
      ↪ $HOME/sfw/linux/silo/4.10.2 --with-zlib="$HOME/sfw/
      ↪ linux/zlib/1.2.11/include,$HOME/sfw/linux/zlib/1.2.11/
      ↪ lib" --disable-silex
make
make install
```

- HDF5 1.10 renamed `--enable-production` to `--enable-build-mode=production`; `make -j4` compiles four files at a time, the first parallel build in these recipes.
- zlib is built for Silo, which is told where its headers and library are (`--with-zlib="include,lib"`).
- The HDF5 source is unpacked in `$HOME/sfw/hdf5` (the shell is still in `$HOME/sfw` after Eigen) and installed in `$HOME/sfw/linux/hdf5/1.10.5`.

"IBAMR Install" (2019), pages 1–2.

DOGWOOD, 2019 (4/9): PETSC 3.10.5, OPTIMIZED AND DEBUG

"IBAMR Install" (2019), pages 2–3 • historical, do not run

```
cd $HOME/sfw
mkdir petsc
cd petsc
wget http://ftp.mcs.anl.gov/pub/petsc/release-snapshots/petsc-3.10.5.
      ↪ tar.gz
tar xvfz petsc-3.10.5.tar.gz
mv petsc-3.10.5 3.10.5
cd 3.10.5
export PETSC_DIR=$HOME/sfw/petsc/3.10.5
export PETSC_ARCH=linux-opt
./config/configure.py \
 --CC=mpicc \
 --CXX=mpicxx \
 --FC=mpif90 \
 --COPTFLAGS="-O3" \
 --CXXOPTFLAGS="-O3" \
 --FOPTFLAGS="-O3" \
 --LDFLAGS="-L/usr/mpi/gcc/openmpi-2.0.3/lib64 -Wl,-rpath,/usr/mpi/
      ↪ gcc/openmpi-2.0.3/lib64" \
 --with-default-arch=0 \
 --PETSC_ARCH=$PETSC_ARCH \
 --with-debugging=0 \
 --with-c++-support \
 --with-hypre=1 \
```

continued; then the debug build • historical, do not run

```
 --download-hypre=1 \
 --with-x=0
make
make test
export PETSC_DIR=$PWD
export PETSC_ARCH=linux-debug
./configure \
 --CC=mpicc \
 --CXX=mpicxx \
 --FC=mpif90 \
 --with-debugging=1 \
 --download-hypre=1 \
 --with-x=0
make
make test
```

- The compilers are the module's MPI wrappers; the `-rpath` points at the module's OpenMPI in `/usr/mpi/gcc/openmpi-2.0.3/lib64`.
- Optimized: `./config/configure.py` with the full 2012-style list; debug: `./configure` with a short one (no `--LDFLAGS`, no `--with-c++-support`).

"IBAMR Install" (2019), pages 2–3.

DOGWOOD, 2019 (5/9): SAMRAI WITH IBAMR-SAMRAI-FIXES.PATCH

"IBAMR Install" (2019), pages 3–4 • historical, do not run

```
cd $HOME/sfw
mkdir samrai
cd samrai
mkdir 2.4.4
cd 2.4.4/
tar xvf $HOME/SAMRAI-v2.4.4.tar
mv SAMRAI/ SAMRAI-2.4.4
cd SAMRAI-2.4.4/
cp $HOME/ibamr-samrai-fixes.patch .
patch -p1 < ibamr-samrai-fixes.patch
cd $HOME/sfw/samrai/2.4.4
mkdir objs-debug
cd objs-debug
```

A new patch file, applied with `patch -p1` (no `gunzip`, no `includes --link`).

the debug configure, broken where the PDF breaks it; then the optimized build • historical, do not run

```
../SAMRAI-2.4.4/configure --prefix=$HOME/sfw/samrai/2.4.4/linux-
      ↪ g++-debug
--with-CC=mpicc --with-CXX=mpicxx --with-F77=mpifort --with-
hdf5=$HOME/sfw/linux/hdf5/1.10.5 --without-petsc --without-hypre
      ↪  --with-
silo=$HOME/sfw/linux/silo/4.10.2 --without-blaslapack --without-
      ↪ cubes --
without-eleven --without-kinsol --without-petsc --without-
      ↪ sundials --
without-x --with-doxygen --with-dot --enable-debug --disable-opt
      ↪  --
enable-implicit-template-instantiation --disable-deprecated
make
make install
cd $HOME/sfw/samrai/2.4.4
mkdir objs-opt
cd objs-opt
# the optimized configure, written with backslashes:
#   CFLAGS, CXXFLAGS, FFLAGS="-O3", prefix .../linux-g++-opt,
#   --with-CC=mpicc --with-CXX=mpicxx --with-F77=mpifort,
#   HDF5 1.10.5, Silo 4.10.2, the list of slide 17
make
make install
```

"IBAMR Install" (2019), pages 3–4.

DOGWOOD, 2019 (6/9): LIBMESH 1.4.1, WITH EXODUSII AND WITHOUT BOOST

"IBAMR Install" (2019), pages 4–5 • historical, do not run

```
mkdir libmesh
cd libmesh
mkdir 1.4.1
cd 1.4.1
wget https://github.com/libMesh/libmesh/releases/download/v1
      ↪ .4.1/libmesh-1.4.1.tar.gz
tar xvfz libmesh-1.4.1.tar.gz
mv libmesh-1.4.1 LIBMESH
cd $HOME/sfw/linux/libmesh/1.4.1
mkdir objs-debug
cd objs-debug
../LIBMESH/configure \
    --prefix=$HOME/sfw/linux/libmesh/1.4.1/1.4.1-debug \
    --with-methods=dbg \
    PETSC_DIR=$HOME/sfw/petsc/3.10.5 \
    PETSC_ARCH=linux-debug \
```

continued • historical, do not run

```
    CC=mpicc \
    CXX=mpicxx \
    FC=mpif90 \
    F77=mpif90 \
    --enable-exodus \
    --enable-triangle \
    --disable-boost \
    --disable-openmp \
    --disable-perflog \
    --disable-pthreads \
    --disable-strict-lgpl \
    --disable-glibcxx-debugging
make
make install
```

`--enable-exodus` for IBFE's ExodusII output, Triangle for 2D meshes, and `--disable-boost`: this libMesh needs no Boost, so the bundled-Boost trick of 2016 is gone. Optimized: `objs-opt`, the prefix `.../1.4.1-opt`, `--with-methods=opt`, `PETSC_ARCH=linux-opt`, the same options; `make`, `make install`.

"IBAMR Install" (2019), pages 4–5.

DOGWOOD, 2019 (7/9): IBAMR V0.4.1, THE DEBUG BUILD

"IBAMR Install" (2019), page 6 • historical, do not run

```
cd $HOME/sfw
mkdir ibamr
cd ibamr
wget https://github.com/IBAMR/IBAMR/archive/v0.4.1.tar.gz
tar xf v0.4.1.tar.gz
mv IBAMR-0.4.1 IBAMR
cd $HOME/sfw/ibamr
mkdir ibamr-objs-dbg
cd ibamr-objs-dbg
export BOOST_ROOT=$HOME/sfw/linux/boost/1.7.0
export PETSC_ARCH=linux-debug
export PETSC_DIR=$HOME/sfw/petsc/3.10.5
../IBAMR/configure \
 CFLAGS="-g -O1 -Wall" \
 CXXFLAGS="-g -O1 -Wall" \
 FCFLAGS="-g -O1 -Wall" \
```

continued • historical, do not run

```
 CC=mpicc \
 CXX=mpicxx \
 FC=mpif90 \
 CPPFLAGS="-DOMPI_SKIP_MPICXX" \
 --with-hypre=$PETSC_DIR/$PETSC_ARCH \
 --with-samrai=$HOME/sfw/samrai/2.4.4/linux-g++-debug \
 --with-hdf5=$HOME/sfw/linux/hdf5/1.10.5 \
 --with-silo=$HOME/sfw/linux/silo/4.10.2 \
 --with-boost=$HOME/sfw/linux/boost/1.7.0 \
 --enable-libmesh \
 --with-libmesh=$HOME/sfw/linux/libmesh/1.4.1/1.4.1-debug \
 --with-libmesh-method=dbg \
--with-zlib=$HOME/sfw/linux/zlib/
make lib
make examples
```

The first recipe built from a tagged IBAMR release, v0.4.1 (a GitHub archive tarball), not from a moving branch: it can be repeated.

DOGWOOD, 2019 (8/9): IBAMR V0.4.1, THE OPTIMIZED BUILD

"IBAMR Install" (2019), pages 6–7 • historical, do not run

```
cd $HOME/sfw/ibamr
mkdir ibamr-objs-opt
cd ibamr-objs-opt
export PETSC_ARCH=linux-opt
export PETSC_DIR=$HOME/sfw/petsc/3.10.5
../IBAMR/configure \
 CC=mpicc \
 CXX=mpicxx \
 F77=mpif90 \
 FC=mpif90 \
 MPICC=mpicc \
 MPICXX=mpicxx \
 CFLAGS="-O2 -funroll-loops -Wall" \
 CXXFLAGS="-O2 -funroll-loops -Wall" \
 FCFLAGS="-O3 -funroll-loops -Wall" \
 CPPFLAGS="-DOMPI_SKIP_MPICXX" \
 --with-hypre=$PETSC_DIR/$PETSC_ARCH \
 --with-samrai=$HOME/sfw/samrai/2.4.4/linux-g++-opt \
 --with-hdf5=$HOME/sfw/linux/hdf5/1.10.5 \
 --with-silo=$HOME/sfw/linux/silo/4.10.2 \
 --with-boost=$HOME/sfw/linux/boost/1.7.0 \
 --enable-libmesh \
 --with-libmesh=$HOME/sfw/linux/libmesh/1.4.1/1.4.1-opt \
 --with-libmesh-method=opt
```

continued • historical, do not run

```
make
make check
```

- `-O2 -funroll-loops` for C and C++, `-O3 -funroll-loops` for Fortran, in place of the `-O3` of earlier recipes (`-funroll-loops`: GCC unrolls loops whose number of iterations it can compute).
- The debug build ran `make lib` and `make examples`; the optimized one `make` and `make check`.
- This is the last recipe in the course's files. The later group builds at Arizona left only their paths (slide 102); since 2025 the series uses autoibamr (chapter 2).

"IBAMR Install" (2019), pages 6–7.

DOGWOOD, 2019 (9/9): WHAT THE PDF HIDES

- Eigen is unpacked but never passed to libMesh or IBAMR (IBAMR then uses its bundled copy).
- `--with-zlib` appears in the debug IBAMR configure only, and names the folder `zlib/`, not `zlib/1.2.11`.
- The PDF wraps long lines where the page ends, often inside an option or a path: the HDF5 configure (`--enable-build-` / `mode=production`), the Silo configure, PETSc's `--LDFLAGS`, the libMesh `wget` address and the debug SAMRAI configure, one long line with no backslashes (`--with-` / `hdf5=...`, `--` / `without-eleven`). Pasted from the PDF, each fails. The SAMRAI line also lists `--without-petsc` twice.
- PETSc's optimized build uses `./config/configure.py`, the debug build `./configure`.
- Boost 1.70.0 sits in a folder named `1.7.0`.
- `mkdir libmesh` comes without a `cd $HOME/sfw/linux`: the shell is still in `samrai/2.4.4/objs-opt`, so the folder is made there, and the next `cd $HOME/sfw/linux/libmesh/1.4.1` fails. The debug libMesh still builds, in that wrong folder, and its `make install` creates `$HOME/sfw/linux/libmesh/1.4.1`; the optimized step then `cd`s there, finds no `../LIBMESH/configure` and fails, and so does the optimized IBAMR that needs it.
- Debug: `make lib`, `make examples`; optimized: `make`, `make check`.
- The PDF's metadata title is its first line, "I used the following library/package versions:".

"IBAMR Install" (2019), checked in October 2026.

THEN AND NOW, SIDE BY SIDE (1/2): THE LIBRARIES

	2012 (lab servers)	2016–17 (“Laura’s Install”)	2019 (Dogwood)	2026 (autoibamr, chapter 2)
MPI	OpenMPI 1.4.3, built	module `openmpi_gcc/4.8.1`	module `openmpi_2.0.3/gcc_4.8.5`	the machine’s MPI (Puma: `gnu13`, `openmpi5`)
arrays	Blitz++ 0.9	Boost 1.60.0 (built), Eigen 3.2.7	Boost 1.70.0 (headers), Eigen 3.3.7 (unused)	Boost 1.91.0, Eigen 3.4.0
HDF5, Silo	1.8.7, 4.8	1.8.16, 4.10	1.10.5, 4.10.2 (+ zlib)	1.12.2, 4.11
PETSc	3.1-p7	3.7.2	3.10.5	3.23.3
SAMRAI	2.4.4 + 100922	2.4.4 + 121212	2.4.4 + `ibamr-samrai-fixes`	IBSAMRAI2 2025.10.29
libMesh	–	1.0.0	1.4.1	1.7.8

The recipes of this chapter; chapter 2; the package list of the WSL test install.

THEN AND NOW, SIDE BY SIDE (2/2): IBAMR AND THE BUILD

	2012 (lab servers)	2016–17 (“Laura’s Install”)	2019 (Dogwood)	2026 (autoibamr, chapter 2)
IBAMR	SVN `trunk`	GitHub `master`	v0.4.1	0.19.0 (Puma: 0.16.0)
build	autotools; debug and optimized	autotools; optimized	autotools; debug and optimized	CMake; one optimized build (`--enable-debugging` for a separate debug install)
how	17 slides of commands (ReadMe slides 15–31), typed or pasted	a six-page document	a seven-page PDF	`./autoibamr.sh -j 8`

The 2026 versions are those of a test install of autoibamr’s default, IBAMR 0.19.0, in WSL on a desktop PC (October 2026); Puma’s install is 0.16.0 (chapters 1–2).

The recipes of this chapter; chapter 2; the package list of the WSL test install.

WHAT AUTOIBAMR DOES THAT THESE RECIPES DID BY HAND

by hand, 2011–2019	autoibamr (chapter 2)
find each library, choose versions that work together (PETSc 3.3-p5 “for the most recent version of the Boyce branch”, PETSc 3.7 for the 2016 master)	downloads, configures, builds and installs IBAMR “together with the libraries it needs”, in matching versions; `--ibamr-version` picks an older IBAMR
download from addresses that moved (Google Code, SourceForge, Bitbucket, LLNL)	the script’s own download list
patch SAMRAI 2.4.4 with the right dated patch	IBSAMRAI2, IBAMR’s patched SAMRAI
write a configure line for each library, twice (debug, optimized), and keep the pairs matched	one build; `--enable-debugging` for a separate debug install in `~/autoibamr-debug`
`LD_LIBRARY_PATH` at every login; `-rpath` by hand	`source .../configuration/enable.sh` sets `IBAMR_ROOT`, `PETSC_DIR` and `PATH`
Boost and Eigen tricks (`include` links, renamed folders)	handled inside the script
compile one file at a time (until 2019’s `make -j4`)	`-j N`
Makefiles that include IBAMR’s `make.inc`	CMake with `find_package(IBAMR)` (chapter 3)

The steps are the same as in 2012; the script performs them. When an install fails today, its log names the step, and that step is one of the slides of this chapter.

THE RECURRING FAILURES, AND HOW TODAY'S TOOLS AVOID THEM

failure	where in this chapter	today
pasted text that is not what it looks like (em dashes, non-breaking spaces, curly quotes, a space after a backslash, soft line breaks)	2012 ReadMe; forces deck; 2016 KillDevil notes; El Capitan; "Laura's Install"	copy commands from plain-text files (this series' listings, scripts and READMEs), in an editor that shows white space; better, run the script
the library path (`LD_LIBRARY_PATH` at every login; `libimf.so`; gcc lost from the path)	THLinux; KillDevil 2013 and 2014	`enable.sh`, and libraries linked with their paths
module clashes (two IBAMR or OpenMPI modules; a module plus a home build)	2014 and 2016–17 KillDevil notes	load the documented set (Puma: `gnu13`, `openmpi5`), then `enable.sh` (chapters 1–2)
debug and optimized builds mixed	every recipe with two builds	one optimized build; a debug install apart
moving branches (`trunk`, `master`, the SVN branches of 2013)	2012–13, 2015–17	tagged releases: `--ibamr-version`
a missing `cd`, a misnamed folder	most recipes	the script does the folders
dead download addresses	every recipe, the 2019 one included (PETSc's old `ftp.mcs.anl.gov` address)	the script's download list, kept up to date by its developers

The slips of each recipe, as this chapter shows them; chapters 1–3.

FROM `make.inc` TO CMAKE, AND THE END OF AUTOTOOLS

- Every recipe here builds IBAMR with autotools (`configure`, `make`), and every course program of 2012–2025 is compiled by a Makefile that includes the install's `config/make.inc`.
- IBAMR 0.8.0 (January 2021): "IBAMR can now be configured, compiled, and installed with CMake instead of with autotools."
- autoibamr builds with CMake and installs no `make.inc`, so an old Makefile stops at its `include` line (chapter 3).
- IBAMR 0.20, in development in 2026, removes the autotools build and stops bundling Boost.

> Porting an old program, then, is two jobs: a `CMakeLists.txt` in place of the Makefile, and the code changes of a decade. Chapter X3 maps both, era by era; chapter 3's porting checklist is the short form.

IBAMR's changelog 0.7.1 to 0.8.0; the code check of October 2026 (IBAMR's development branch); chapters 3 and X3.

THE LAST HAND-BUILT INSTALLS, 2020–2025

when, where	install	what the sources say
2020, Dogwood	module `ibamr/0.6.0`	a cluster module; “I recommend installing your own updated version of ibamr as well”
2020–21, Arizona	`/contrib/<group>/ibamr/0.7.1`	a group build of IBAMR 0.7.1 (the UA rubber band and a few other UA folders)
2022–24, Arizona (Ocelote era)	`/groups/<group>/ib10/ibamr/ibamr-objs-opt`	an autotools build used by almost all UA folders; presumably 0.10, judged by the name
July 2025, Arizona	`/groups/<group>/ib16/Code/ibamr-objs-opt`	an autotools build, presumably 0.16; its Makefile builds IBAMR’s own `example.cpp`
2025–26, Puma; a desktop PC (WSL)	autoibamr: IBAMR 0.16.0 on Puma, 0.19.0 by default	this series (chapters 2–3)

The build recipes of the Arizona installs are not in the course’s files; only the paths in Makefiles remain (chapter X3 dates code by them). The clusters themselves, from Kure to Puma, are chapter X1’s subject.

IBFE lecture 5 (June 2020); the Makefiles of `examples/legacy`; the code check of October 2026; chapters 2, X1 and X3.

EVERY RECIPE IN THIS CHAPTER

recipe	date	machine	written by	slides
IBAMR ReadMe, slides 15–31	2012	lab servers; KillDevil	the Miller lab (author not verified)	10, 13–25
forces install notes, “InstallNotes2”, forces deck	Dec 2012 – Jan 2013	lab; KillDevil	the Miller lab (author not verified)	26–32
MATH 769 “IBFE install”	April 2013	lab servers	the instructor	33–35
“THLinuxInstall”	April 2013	THLinux	the instructor’s course folder	36
“Install IBAMR on KillDevil”	2012–13	KillDevil	the instructor	37–42
“Install THLinux and IBAMR”; “Running IBAMR on THLinux / KillDevil”	2011–15	THLinux; KillDevil	the instructor	11, 12, 43–44
December 2013 working notes; “Notes on THLinux”	Dec 2013	THLinux; a developer’s Mac	the instructor, with Boyce Griffith’s notes	11, 45–50
“IBAMR Killdevil Module”; “Using IBAMR Module on Kure”; READ ME	2014	KillDevil, Kure	the instructor; an undergraduate guide (author not recorded)	51–52
“Installing IBAMR on Killdevil using Modules”	2015	KillDevil	Alex Hoover	53–54
OS X Mavericks notes	about 2015	Mac OS X 10.9.5	unattributed	55–60
“July 18 Notes on building IBAMR”	2016	–	the instructor’s notes of Boyce Griffith’s e-mails	61
“IBAMR Installation 4/16/16”	Apr–Aug 2016	KillDevil	Nick Battista	62–67
“IBAMR install on OS X El Capitan”	Aug 2016	Mac OS X 10.11.6	unattributed	70–76
“Laura’s Install” (and “IBAMR Install Notes”); Install Files	Sep 2016 – Apr 2017	KillDevil	the instructor, with Boyce Griffith’s Boost workaround	77–85
“How to update examples to new install”; to-do list; IBFE read-me	2016–17	KillDevil modules	the instructor (code edits by Nick Battista)	68–69
“Run your IBAMR simulation on Dogwood”; IBFE lecture 5	2018–20	Dogwood	the instructor	87
“IBAMR Install” (PDF)	2019	Dogwood	not stated	88–96

The recipes themselves, compared in October 2026. Where a recipe’s author is not credited by the series, it stays unattributed.

SUMMARY

- From 2011 to 2019 an IBAMR install was eight to ten libraries built by hand in a fixed order, PETSc, SAMRAI, libMesh and IBAMR usually twice; the versions moved every year or two (slides 5–6).
- The same few slips recur in every era: pasted characters that are not what they look like, a missing `cd`, a renamed folder, the library path, a debug build mixed with an optimized one, a moving branch.
- December 2013 (Blitz++ to Boost and Eigen), 2016 (PETSc 3.7, libMesh 1.0, an external Boost) and 2019 (a tagged release, v0.4.1) are the turning points; each also changed the course code (chapter X3).
- autoibamr (chapter 2) does every step of these recipes, with versions that match.

Keep from them: write install notes in plain text, with the version of every library; pin releases, not branches; build in a folder beside the source; test each library before the next (`make check`, `make test`); and never paste a command from a word processor.

This chapter.